UNITY GAME DESIGN DOCUMENT

2D Game Kit

The Explorer: 2D

"The Explorer: 2D is a collection of mechanics, tools, systems and assets to hook up gameplay without writing any code."

— Unity Technologies

원본 제작	Unity Technologies — 2D Game Kit (Unity Learn)
문서 작성	원광대학교 게임콘텐츠학과
과목	게임프로그래밍 강의자료
작성 근거	C# 스크립트 126개(11,552줄) 전수 분석 + Unity Learn 공식 학습자료 5종
버전 / 작성일	v1.0 / 2026. 10. 07.

목차

0. 문서 정보 4
1. 개요 (Overview) 4
    1-1. 핵심 특징 5
    1-2. "코드를 쓰지 않는다"는 말의 의미 5
2. 학습 과정으로서의 구성 6
    2-1. Walkthrough 10단계 7
3. 게임 개요 (Game Overview) 8
    3-1. 이야기 배경 8
    3-2. 세계와 씬 구성 9
    3-3. 승리 조건과 패배 조건 9
4. 게임플레이 (Gameplay) 10
    4-1. 조작 10
    4-2. Ellen의 능력 11
    4-3. 전투 규칙 12
    4-4. 난이도 요소 14
5. 캐릭터 (Characters) 14
    5-1. 등장 캐릭터 일람 15
    5-2. Ellen 15
    5-3. Chomper와 Spitter — 같은 컴포넌트, 다른 조종 주체 17
    5-4. Gunner (MissileGolem) — 보스 설계 19
6. 레벨과 환경 (Levels & Environment) 20
    6-1. 타일맵과 타일셋 20
    6-2. 상호작용 오브젝트 21
    6-3. 이동 플랫폼의 경로 처리 23
7. 시스템 설계 (System Design) 23
    7-1. 입력 — PlayerInput의 2단계 차단 구조 23
    7-2. 캐릭터 이동과 지면 판정 24
    7-3. 피해 시스템 — Damager / Damageable 26
    7-4. 애니메이터 연동 — SceneLinkedSMB 26
    7-5. 적 AI — 두 가지 방식 27
    7-6. 오브젝트 풀 29
    7-7. 상호작용과 인벤토리 30
    7-8. 씬 전환과 데이터 영속화 32
    7-9. 소리와 이펙트 34
    7-10. 다국어와 타임라인 35
8. 기술 명세 (Technical Specification) 36
    8-1. 환경과 네임스페이스 36
    8-2. 디렉터리 구조와 코드 규모 37
    8-3. 주요 수치 — 코드 기본값과 Reference Guide 권장값 38

8-4. enum 식별자 전수 42
9. 알려진 제약과 주의점 44
9-1. 동작하지 않거나 쓰이지 않는 코드 44
9-2. 이름이나 문서와 실제 동작이 다른 것 45
9-3. 설정이나 상황에 따라 문제가 되는 것 46
10. 확장 가이드 47
10-1. 새 적 추가 47
10-2. 새 상호작용 오브젝트 추가 48
10-3. 새 오브젝트 풀 만들기 50
10-4. 그 밖의 확장 지점 51
11. 참고 자료 52
11-1. Unity Learn 공식 자료 52
11-2. 프로젝트 내부 자료 52
11-3. 문서 작성 기준 53

0. 문서 정보

이 문서는 Unity Technologies가 배포하는 학습용 프로젝트「2D Game Kit (The Explorer: 2D)」을 Unity Learn Game Development Pathway의 GDD 템플릿 구조에 맞춰 정리한 강의자료입니다. 게임 디자인 문서의 형식을 빌려 "완성된 2D 액션 어드벤처가 내부에서 어떻게 조립되어 있는가"를 기술하는 것이 목적이며, 다음 두 종류의 근거만을 사용했습니다.

분석 자료	내용	반영 섹션
프로젝트 C# 소스	Assets/2DGameKit/Scripts 전체 — 126개 파일, 11,552줄. 컴포넌트 구조, 직렬화 필드의 코드 기본값, enum 식별자, 실행 순서, 이벤트 연결 방식	4 ~ 10
Unity Learn 공식 자료	① Get Started with the 2D Game Kit ② 2D Game Kit Walkthrough ③ Video: 2D Game Kit Walkthrough ④ 2D Game Kit Reference Guide ⑤ 2D Game Kit Advanced Topics	1 ~ 3, 6, 9, 11

이 문서가 수치를 적는 원칙

· 모든 수치·필드명·클래스명·enum 식별자는 소스코드에서 직접 확인한 값만 적었다. 확인되지 않은 값은 적지 않고 "코드에 초기화 없음"이라고 표시했다.

· 같은 항목에 대해 코드의 기본값과 Reference Guide가 제시하는 Ellen 프리팹 권장값이 다른 경우가 많다. 8장의 수치 표는 두 값을 나란히 두고 차이를 명시한다.

· 코드 기본값은 "새 컴포넌트를 붙였을 때의 값"이고, 권장값은 "완성된 Ellen 프리팹에 저장되어 있는 값"이다. 둘을 혼동하면 직접 만든 캐릭터가 공식 Ellen과 다르게 움직인다.

1. 개요 (Overview)

제목	2D Game Kit — 부제 "The Explorer: 2D"
한 문장 요약 (High Concept)	외계 행성에 불시착한 탐사자 Ellen이, 고대 외계 문명이 남긴 폐허와 지하 묘소를 건너며 적대적인 생물과 환경 위험을 뚫고 길을 여는 2D 액션 어드벤처 — 그리고 그 게임을 코드 한 줄 쓰지 않고 조립하는 방법을 가르치는 학습 키트.
핵심 동사	달리다 → 뛰어오르다 → 베다 / 쏘다 → 밀다 → 밟다 → 열다 → 건너가다
장르	2D 사이드뷰 액션 플랫포머 / 탐사형 어드벤처 (메트로배니아 성향의 구역 연결 구조)

플레이 인원 / 시점	1인 / 2D 사이드뷰, Cinemachine 기반 카메라 추적
대상 플랫폼	Unity가 지원하는 전 플랫폼 (키보드 + 게임패드 입력을 모두 바인딩)
Unity 버전	Unity 2017.3 이상 (Unity Learn 공식 요구 사항)
학습 난이도 / 소요 시간	Beginner / 4시간 25분 (Unity Learn 공식 표기)
대상 학습자	Unity 에디터 조작은 익혔으나 아직 스크립트를 자유롭게 쓰지 못하는 입문자. 프리팹·인스펙터·UnityEvent·Animator로 게임플레이를 조립하는 감각을 기르는 단계

1-1. 핵심 특징

- **완성된 게임이 통째로 들어 있다:** 예제 조각이 아니라 Zone1~Zone5 의 5 개 레벨과 보스전까지 플레이할 수 있는 한 벌의 게임이 들어 있다. 학습자는 빈 화면에서 시작하지 않고, 이미 돌아가는 게임을 분해하고 변형하면서 배운다.
- **조립식 게임플레이:** 문을 여는 일은 "압력판이 눌리면 문 애니메이터의 DoorOpening 을 재생한다"는 인스펙터 연결 하나로 끝난다. 조건과 결과를 UnityEvent 로 잇는 방식이 키트 전체를 관통한다.
- **타일맵 기반 레벨 제작:** Tile Palette 로 지형을 그리면 콜라이더·발소리·먼지 이펙트가 타일 종류에 따라 자동으로 달라진다.
- **두 가지 적 AI 패러다임을 한 프로젝트에서 비교할 수 있다:** Chomper 는 Animator 상태 기계로, Spitter 와 보스 Gunner 는 행동 트리(BTAI)로 조종된다. 같은 EnemyBehaviour 를 쓰면서 조종 주체만 다르다.
- **성능을 의식한 기본 구조:** 총알·적·이펙트는 모두 오브젝트 풀에서 꺼내 쓰고 반납한다. 입문 키트이면서도 Instantiate/Destroy 를 반복하지 않는 패턴을 기본값으로 보여 준다.

1-2. "코드를 쓰지 않는다"는 말의 의미

공식 소개 문구의 "without writing any code"는 이 프로젝트에 코드가 없다는 뜻이 아닙니다. 정반대로, 학습자가 쓸 일이 없도록 코드가 미리 다 쓰여 있다는 뜻입니다. 이 구분이 이 GDD의 출발점입니다.

구분	학습자가 하는 일	코드가 이미 해 둔 일
레벨	Tile Palette로 타일을 칠한다	타일맵의 콜라이더, 타일별 발소리 오버라이드 (RandomAudioPlayer), 먼지 이펙트 선택 (VFXController)

구분	학습자가 하는 일	코드가 이미 해 둔 일
이동 플랫폼	경로 노드를 드래그하고 Speed와 Looping을 고른다	노드 간 보간, BACK_FORTH/LOOP/ONCE 분기, 플랫폼 위 물체를 함께 옮기는 PlatformCatcher
문과 압력판	On Pressed에 Animator.Play("DoorOpening")을 연결한다	접촉점 법선으로 "위에서 누르는 것"만 세는 판정, 2프레임 지연 확정, OnPressed/OnRelease 발생
적 배치	Chomper·Spitter 프리팹을 끌어다 놓고 speed·FOV·health를 채운다	시야 판정, 추격·공격·사망 상태 전이, 탄도 계산, 피격 점멸, 오브젝트 풀 반납
구역 이동	TransitionPoint와 Destination을 놓고 태그를 맞춘다	저장 → 입력 차단 → 페이드 → 비동기 로드 → 복원 → 배치 → 페이드인의 14단계 코루틴

따라서 이 키트를 교재로 쓸 때의 학습 목표는 두 단계로 나뉩니다. 1단계는 "조립해서 게임을 만든다"이고, 2단계는 "조립 뒤에 숨은 11,552줄이 무엇을 하고 있는지 읽는다"입니다. 이 문서의 7장 이후는 2단계를 위한 것입니다.

2. 학습 과정으로서의 구성

Unity Learn의 "2D Game Kit" 과정(https://learn.unity.com/course/2d-game-kit)은 다섯 개의 튜토리얼로 이루어져 있습니다. 난이도 Beginner, 총 소요 시간 4시간 25분, Unity 2017.3 이상을 요구합니다.

No.	튜토리얼	다루는 범위	이 문서의 대응 절
1	Get Started with the 2D Game Kit	키트 내려받기, 프로젝트 폴더 구조, ExampleScene 열어 보기, 무엇이 들어 있는지 개괄	1, 6
2	2D Game Kit Walkthrough	새 씬 만들기부터 타일 칠하기·이동 플랫폼·문과 압력판·적 배치·피해 주는 오브젝트·장식·순간이동까지 10단계 실습	3 ~ 6
3	Video: 2D Game Kit Walkthrough	위 10단계를 8개 장으로 나눈 45분 영상 강의	3 ~ 6

No.	튜토리얼	다루는 범위	이 문서의 대응 절
4	2D Game Kit Reference Guide	Ellen · Enemy Behaviour · Health Pickup · Pressure Pad · Damage system · Moving Platform · Interaction system · Inventory system · Hub Door · Enemy Spawner 10개 절의 컴포넌트 설명과 권장 설정값	5 ~ 8
5	2D Game Kit Advanced Topics	RandomAudioPlayer · VFXController · Data Persistence · SceneLinkedSMB · Object Pooling · Behaviour Tree 6개 절의 내부 구조 설명	7, 10, 11

2-1. Walkthrough 10 단계

두 번째·세 번째 튜토리얼이 공통으로 따라가는 실습 순서입니다. 이 순서가 곧 "키트로 한 레벨을 만드는 표준 절차"이므로, 수업에서는 이 10단계를 그대로 1회차 실습으로 쓸 수 있습니다.

단계	작업	쓰는 메뉴 · 에셋	확인할 점
①	Creating a New Scene	Kit Tools > Create New Scene	키트가 필요한 매니저·UI·카메라를 갖춘 씬을 자동으로 구성한다. 빈 씬에서 시작하지 않는다.
②	Painting a Level	Window > 2D > Tile Palette, Tileset_Gamekit	타일셋은 Grassy Rocks와 Alien Structure 두 가지. 지상 구역과 지하 유적의 분위기를 가른다.
③	Testing Your Level	Play	F12로 근접/원거리 무기 사용 가능 여부를 토글해 테스트할 수 있다.
④	Adding a Moving Platform	Prefabs > Interactables	경로가 빨간 점선으로 보인다. Preview Position 슬라이더로 미리 움직여 보고, Add Node로 노드를 늘린다. Looping을 BACK_FORTH에서 Loop로 바꾸면 순환 경로가 된다. Speed 기본 2.
⑤	Opening a Door with Events	Door 프리팹 + PressurePad 프리팹	PressurePad의 On Pressed에 문의 Animator > Play(string)을 연결하고 문자열 "DoorOpening"을 적는다. 코드를 쓰지 않는 게임플레이 연결의 대표 예.

단계	작업	쓰는 메뉴 · 에셋	확인할 점
⑥	Enemies	Prefabs > Enemies (Chomper, Spitter)	Enemy Behaviour의 speed · FOV · health를 채운다. Spitter 예시값은 View Direction 30 / View Fov 182 / View Distance 6.
⑦	Damaging with Objects	PushableBox + Damager 컴포넌트 추가	초록 박스가 피해 판정 영역이다. Hittable Layers를 Enemy로 지정하면 밀어서 적을 죽이는 상자가 된다.
⑧	Decorating	Art > Sprites > Environment	타일맵 위에 스프라이트를 얹어 분위기를 만든다. 게임플레이에 영향이 없는 순수 장식 단계.
⑨	Teleporting the Player	Prefabs > SceneControl (TransitionStart / TransitionEnd / TransitionDestination)	Destination Tag를 서로 맞추고 Transition Type을 Same Scene(같은 씬 안 이동) 또는 Different Zone(다른 구역)으로 고른다.
⑩	Having Fun	Scenes > Start, Zone1 ~ Zone5	완성품을 처음부터 끝까지 플레이해 자신이 만든 레벨과 비교한다.

3. 게임 개요 (Game Overview)

3-1. 이야기 배경

주인공은 탐사자 Ellen입니다. Ellen은 외계 행성에 불시착해, 고대 외계 문명이 남긴 위험한 폐허를 헤치고 지하 묘소(underground crypts)에 이릅니다. 그 길에는 적대적인 생물과 환경 위험이 놓여 있고, Ellen은 이들을 통과해야 합니다. 이야기는 대사나 컷신으로 길게 설명되지 않고, 지형과 적의 배치, 그리고 잠긴 문 앞에서 열쇠를 찾아 돌아오는 구조로 전달됩니다.

이야기가 메커닉에 남긴 흔적

· "불시착"은 Ellen이 무장하지 않은 채 시작하는 이유다. 무기는 레벨 안의 WeaponPickup으로 얻고, 획득 여부는 PlayerInput이 IDataPersister로 저장해 구역을 옮겨도 유지된다.

· "고대 문명의 폐허"는 두 타일셋(Grassy Rocks / Alien Structure)의 대비로 표현된다. 지상의 풀과 바위에서 외계 구조물로 들어가는 진행이 곧 이야기의 진행이다.

· "환경 위험"은 적과 다른 계통의 위협이다. 물(WaterArea)이나 함정도 Damager로 피해를 주며, forceRespawn을 켜 두면 체력을 깎는 동시에 체크포인트로 강제 복귀시킨다.

3-2. 세계와 씬 구성

씬	역할	수업에서의 쓰임
ExampleScene	키트의 요소를 한자리에 모아 둔 학습용 씬	첫 수업에서 각 프리팹이 무엇을 하는지 관찰하는 데 쓴다
Start	게임 전체의 시작 지점	타이틀·시작 흐름과 SceneController의 초기 목적지 설정을 확인한다
Zone1 ~ Zone5	사전 제작된 플레이 가능 레벨 5개. 보스 Gunner는 Zone5에 있다	완성된 레벨의 구성과 난이도 배치를 분석하는 교재
Template	새 레벨을 만들 때 복제해 쓰는 빈 템플릿 씬	학생 과제의 출발점
UIMenus	일시정지 메뉴. 플레이 중 Additive로 겹쳐 로드된다	일시정지 구현이 씬 단위로 처리되는 방식을 보여 준다

구역 사이의 연결은 메트로배니아 성향을 띱니다. 한 구역에서 얻은 열쇠가 다른 구역의 Hub Door를 열고, TransitionPoint를 통과하면 그 구역의 상태가 저장된 채 다음 씬이 비동기로 로드됩니다. 플레이어가 "되돌아간다"는 느낌을 받을 수 있도록, 적의 체력·죽은 적·먹은 아이템·열쇠 소지 상태가 모두 영속화 대상입니다(7-8절).

3-3. 승리 조건과 패배 조건

구분	조건	처리 주체
진행	각 구역의 TransitionPoint에 도달해 다음 구역으로 넘어간다. 잠긴 문은 해당 열쇠를 모아야 열린다.	TransitionPoint, HubDoor
최종 승리	Zone5의 보스 Gunner(MissileGolem)를 rounds 배열에 정의된 라운드 수만큼 모두 격파한다. 마지막에 onDefeated 이벤트가 발생한다.	MissileGolem
피격	Damager의 판정 영역에 닿으면 체력이 damage(코드 기본값 1)만큼 줄고, 피격 방향으로 튕겨 나가며 invulnerabilityDuration(코드 기본값 3초) 동안 무적이 된다.	Damager → Damageable
사망	체력이 0 이하가 되면 Damageable.OnDie가 발생한다. Ellen은 사망 연출 후 마지막 체크포인트에서 리스폰한다.	Damageable, PlayerCharacter
강제 리스폰	forceRespawn이 켜진 Damager에 맞으면 체력이 남아 있어도 ForcedRespawn 트리거로 체크포인트로 되돌아간다.	Damager, PlayerCharacter

게임 오버가 아니라 되돌아가기

· 이 키트에는 "게임 오버 후 처음부터"가 없다. 죽으면 마지막 Checkpoint로 돌아가고, 적의 체력 등은 SceneController가 구역 재시작 정보(m_ZoneRestartDestinationTag)를 들고 있어 그 구역 입구부터 다시 시작된다.

· 이 설계는 입문 학습자가 실수로 전체 진행을 잃지 않게 하려는 교육적 선택이다. 난이도를 올리려면 Checkpoint 간격을 넓히는 것이 가장 직접적인 수단이 된다.

4. 게임플레이 (Gameplay)

4-1. 조작

입력은 PlayerInput 컴포넌트가 Unity의 새 입력 시스템(UnityEngine.InputSystem)으로 정의합니다. 각 항목은 키보드 바인딩과 게임패드 바인딩을 동시에 들고 있어, 두 장치가 별도 설정 없이 함께 동작합니다. 아래 값은 PlayerInput.cs의 필드 초기화에서 그대로 읽은 것입니다.

동작	필드명	키보드	게임패드	비고
좌우 이동	Horizontal	D (+) / A (−)	왼쪽 스틱 좌우	1DAxis 복합 바인딩. Value는 −1 ~ 1
상하 입력	Vertical	W (+) / S (−)	왼쪽 스틱 상하	아래는 웅크리기, 위는 카메라 올려 보기
점프	Jump	Space	buttonSouth	가변 점프 — 버튼을 일찍 떼면 상승 속도가 깎인다
근접 공격	MeleeAttack	K	buttonWest	무기를 얻기 전에는 Disable 상태
원거리 공격	RangedAttack	O	buttonEast	무기를 얻기 전에는 Disable 상태
상호작용	Interact	E	buttonNorth	InteractOnButton2D 영역 안에서만 유효
일시정지	Pause	Escape	start	timeScale 0 + UIMenus 씬 Additive 로드
디버그 메뉴	(PlayerInput 내부)	F12	—	근접/원거리 공격 허용 여부를 켜고 끄는 OnGUI 창

조작 안내 문구는 바인딩에서 만들어진다

· UI/InputDisplayUI.cs는 F12 창이 아니다. OnEnable에서 PlayerInput의 각 InputAction에 걸린 키보드 바인딩 경로를 읽어, "Space - Jump" 같은 안내 문구를 StringBuilder로 조립해 TextMeshProUGUI에 넣는 UI다.

· 즉 키 바인딩을 바꾸면 조작 안내도 함께 바뀐다. 다만 OnEnable에서 한 번만 만들므로, 실행 중에 바인딩을 바꾸면 갱신되지 않는다.

· 바인딩을 찾지 못하면 "Unbound"를, PlayerInput.Instance가 없으면 "## ERROR ## No PlayerInput detected"를 표시한다.

4-2. Ellen 의 능력

능력	입력	규칙 (코드 확인)	관련 함수
걷기 · 달리기	Horizontal	목표 속도는 maxSpeed × 입력값. 입력이 있으면 groundAcceleration, 없으면 groundDeceleration으로 MoveTowards 보간한다.	GroundedHorizontalMovement
점프	Space	SMB가 SetVerticalMovement(jumpSpeed)를 호출해 상승 속도를 준다.	CheckForJumpInput
가변 점프	Space 떼기	버튼을 일찍 떼면 jumpAbortSpeedReduction만큼 상승 속도를 깎는다. 짧게 누르면 낮게 뛴다.	AirborneSMB
웅크리기	S	Vertical.Value < 0이면 Crouching 파라미터가 true가 된다.	CheckForCrouching
플랫폼 통과	S + Space	밟고 있는 바닥 콜라이더 3개가 전부 PlatformEffector2D를 가질 때만 성립한다. 통과하는 동안 0.5초 무적을 걸어, 빠져나오며 다시 서는 과정에서 피해를 입지 않게 한다.	MakePlatformFallthrough
근접 공격	K	MeleeAttack 트리거 → meleeDamager가 활성화되고, 바라보는 방향(±1)으로 meleeAttackDashSpeed만큼 미끄러진다. dashWhileAirborne이 false이면 공중에서는 미끄러지지 않는다.	MeleeAttack, EnableMeleeAttack

능력	입력	규칙 (코드 확인)	관련 함수
원거리 공격	O	shotsPerSecond의 역수가 발사 간격(m_ShotSpawnGap)이 된다. 총알은 BulletPool에서 꺼내고 bulletSpeed로 날아간다. 쏘지 않으면 holdingGunTimeoutDuration 후 들기 자세가 풀린다.	SpawnBullet, CheckForHoldingGun
밀기	상자에 붙어 이동	트리거에 겹친 Pushable 중 하나를 고르고, 최대 속도에 pushingSpeedProportion을 곱해 느려진다.	CheckForPushing
상호작용	E	InteractOnButton2D 영역 안에서 Interact.Down이면 OnButtonPress가 발생한다.	InteractOnButton2D.Update

4-3. 전투 규칙

공격과 피격은 모두 Damager → Damageable 한 쌍으로 처리됩니다. 공격하는 쪽이 Damager, 맞는 쪽이 Damageable이며, 플레이어와 적이 같은 규칙을 공유합니다.

항목	규칙	코드 기본값
피해량	Damager.damage만큼 Damageable의 체력이 줄어든다	damage = 1
판정 영역	Damager의 offset·size로 정한 직사각형. 인스펙터에서 초록 박스로 보인다	offset (1.5, 1) / size (2.5, 1)
방향 반영	offsetBasedOnSpriteFacing이 켜져 있으면 스프라이트가 뒤집힐 때 offset.x의 부호도 뒤집혀, 판정 영역이 항상 바라보는 쪽에 생긴다	true
대상 제한	hittableLayers에 포함된 레이어만 맞는다. 상자에 Damager를 달 때 Hittable Layers를 Enemy로 지정하는 것이 이것이다	(초기화 없음 — 반드시 지정)
시작 체력	Damageable.startingHealth. OnEnable 때마다 이 값으로 되돌아간다	startingHealth = 5
피격 후 무적	invulnerableAfterDamage가 켜져 있으면 invulnerabilityDuration 동안 무적. 그 사이 TakeDamage는 조용히 반환한다	true / 3초
무적 관통	Damager.ignoreInvincibility가 켜져 있으면 무적 상태에서도 OnTakeDamage 콜백은 발생한다. 단 체력은 줄지 않는다	false

항목	규칙	코드 기본값
1회성 피해	disableDamageAfterHit이 켜져 있으면 한 번 맞히고 스스로 DisableDamage()한다. 총알·수류탄에 쓴다	false
사망 처리	체력이 0 이하가 되면 OnDie 발생 → 영구 무적 전환 → disableOnDeath면 오브젝트 비활성화	disableOnDeath = false
피격 방향	(자기 위치 + centreOffset) − Damager 위치. 이 벡터로 튕겨 나가는 방향을 정한다	centreOffset (0, 1)
피격 반동	피격 시 hurtJumpAngle 방향으로 hurtJumpSpeed만큼 튕긴다. 각도는 Start()에서 0.001~89.999도로 제한된 뒤 탄젠트로 미리 변환된다	45도 / 5
점멸	피격 후 flickeringDuration 간격으로 SpriteRenderer를 껐다 켠다	0.1초

피해 전달의 호출 순서

```
Damager.FixedUpdate()
  └ Physics2D.OverlapArea(pointA, pointB, m_AttackContactFilter, m_AttackOverlapResults)
      └ 걸린 콜라이더마다
          ├ Damageable 없음  →  OnNonDamageableHit.Invoke(this)
          └ Damageable 있음  →  OnDamageableHit.Invoke(this, damageable)
                                damageable.TakeDamage(this, ignoreInvincibility)
                                  ├ (무적 && !ignore) 또는 체력 0 이하  →  즉시 반환
                                  ├ m_CurrentHealth -= damager.damage  →  OnHealthSet
                                  ├ OnTakeDamage.Invoke(damager, this)
                                  └ 체력 0 이하  →  OnDie.Invoke()
                                                    EnableInvulnerability()
                                                    disableOnDeath 면 SetActive(false)
                                disableDamageAfterHit 면 DisableDamage()
```

OnTakeDamage / OnDie는 인스펙터에서 이어야 한다

· PlayerCharacter.OnHurt(Damager, Damageable)와 PlayerCharacter.OnDie()는 프로젝트 전체에서 코드가 호출하는 곳이 한 군데도 없다(확인: 두 메서드 선언부 외 참조 0건).

· 즉 Ellen의 피격 연출과 사망 연출은 Damageable의 OnTakeDamage / OnDie 이벤트에 인스펙터에서 연결해 두었을 때만 동작한다. PlayerCharacter.cs의 주석도 이 점을 경고로 남겨 두었다.

· 적도 같다. EnemyBehaviour.Hit(Damager, Damageable)과 Die(Damager, Damageable)가 같은 방식으로 연결된다. 새 적을 만들 때 가장 자주 빠뜨리는 연결이 바로 이것이다.

4-4. 난이도 요소

요소	조절 지점	효과
체력	Damageable.startingHealth	코드 기본값 5. 몇 대까지 버티는지를 정한다. HealthUI가 하트 아이콘 수로 표시한다
회복 간격	HealthPickup.healthAmount	코드 기본값 1. 회복량이 작으면 피격 한 번의 무게가 커진다
체크포인트 간격	Checkpoint 배치	죽었을 때 되돌아가는 거리. 이 키트에서 난이도를 올리는 가장 직접적인 수단
적 시야	viewDirection / viewFov / viewDistance	넓고 멀면 몰래 지나가기 어려워진다. Spitter 예시값은 30 / 182 / 6
적 추격 지속	timeBeforeTargetLost	코드 기본값 3초. 시야에서 벗어나도 이 시간 동안은 쫓는다
적 소환	EnemySpawner.totalEnemiesToBeSpawned / concurrentEnemiesToBeSpawned / spawnDelay	동시에 몇 마리, 총 몇 마리를 어느 간격으로 내보낼지 정한다
환경 위험	Damager.forceRespawn	체력만 깎는 함정과, 체크포인트로 되돌리는 함정을 구분한다
보스 라운드	MissileGolem.rounds[]	라운드마다 bossHP(기본 10) · shieldHP(기본 10) · platformSpeed(기본 1)를 따로 정한다. 라운드 수 자체가 난이도다
보스 조준	laserTrackingSpeed	코드 기본값 30. 빔이 플레이어를 따라 도는 각속도. 작으면 피할 틈이 커진다

5. 캐릭터 (Characters)

5-1. 등장 캐릭터 일람

캐릭터	역할	행동	핵심 스크립트
Ellen	플레이어. 태그 "Player"	이동·점프·웅크리기·플랫폼 통과·근접/원거리 공격·상자 밀기·상호작용. 죽으면 체크포인트에서 리스폰	PlayerCharacter CharacterController2D Damageable, Damager PlayerInput, InventoryController BulletPool
Chomper	근접 적	순찰하다 장애물을 만나면 방향을 되돌린다. 플레이어를 발견하면 달려와 meleeRange 안에서 물고, attackDash로 앞으로 돌진한다	EnemyBehaviour + Animator SMB 4종 (ChomperPatrolSMB, ChomperRunToTargetSMB, ChomperAttackSMB, ChomperDeathSMB)
Spitter	원거리 적	제자리에서 시야를 훑다가 플레이어를 발견하면 마지막으로 본 위치를 기억해 탄도를 계산하고 발사체를 뱉는다	EnemyBehaviour + SpitterBT (행동 트리) Bullet / BulletPool
Gunner	보스. Zone5	라운드마다 쉴드를 올리고, 쉴드가 깨질 때까지 걷기·빔·번개·수류탄 네 패턴을 가중치 추첨으로 번갈아 쓴다. 쉴드가 깨지면 무적이 풀려 본체를 때릴 수 있다	MissileGolem (BTAI) GunnerBeam, GunnerProjectile GunnerLightning, GunnerShieldEffect Grenade

5-2. Ellen

Ellen은 직접 상태를 전이시키지 않습니다. PlayerCharacter는 이번 프레임의 이동 벡터 m_MoveVector를 계산해 두기만 하고, "지금 무엇을 할지"는 Animator의 상태 기계가 정합니다. 각 상태에 붙은 SceneLinkedSMB<PlayerCharacter>가 PlayerCharacter의 public 메서드를 거꾸로 호출하는 구조입니다. 이 역전된 제어 흐름이 Ellen을 읽기 어렵게 만드는 동시에, 상태별 동작을 코드 수정 없이 Animator에서 재배치할 수 있게 합니다.

한 프레임의 처리 순서

```
1. Animator 가 현재 상태의 SMB.OnSLStateNoTransitionUpdate 를 부른다
     → SMB 가 PlayerCharacter 의 메서드를 호출해 m_MoveVector 를 만든다
       (UpdateFacing, GroundedHorizontalMovement, CheckForGrounded,
        CheckForJumpInput, CheckForCrouching, CheckForPushing, ...)


2. PlayerCharacter.Update()   — Pause 버튼만 본다


3. PlayerCharacter.FixedUpdate()
     m_CharacterController2D.Move(m_MoveVector * Time.deltaTime);
     m_Animator.SetFloat(HorizontalSpeed, m_MoveVector.x);
     m_Animator.SetFloat(VerticalSpeed,   m_MoveVector.y);
     UpdateBulletSpawnPointPositions();
     UpdateCameraFollowTargetPosition();
```

Animator 파라미터 12 개

PlayerCharacter가 Animator.StringToHash로 미리 해시해 두고 쓰는 파라미터입니다. 문자열을 매 프레임 해시하지 않기 위해 readonly 필드로 캐시합니다.

파라미터	종류	기록하는 곳	의미
HorizontalSpeed	Float	FixedUpdate	m_MoveVector.x — 달리기 모션 블렌딩
VerticalSpeed	Float	FixedUpdate	m_MoveVector.y — 상승/하강 모션 구분
Grounded	Bool	CheckForGrounded	접지 여부. 지상 ↔ 공중 전이의 기준
Crouching	Bool	CheckForCrouching	Vertical 입력이 아래쪽인지
Pushing	Bool	CheckForPushing	상자를 밀고 있는지
Timeout	Bool	CheckForHoldingGun	총 들기 자세의 유지 시간이 끝났는지
HoldingGun	Bool	CheckForHoldingGun	총을 든 자세인지 — 이동·공중 모션이 별도 세트로 갈린다
MeleeAttack	Trigger	MeleeAttack	근접 공격 모션 진입
Hurt	Trigger	OnHurt	피격 모션 진입. 동시에 Grounded를 false로 만든다
Dead	Trigger	OnDie	사망 모션 진입
ForcedRespawn	Trigger	OnHurt	체력이 남아 있지만 체크포인트로 되돌릴 때
Respawn	Trigger	Respawn	리스폰 모션 진입

Ellen 이 인스펙터에서 반드시 받아야 하는 참조

아래 항목들은 코드에서 null 검사 없이 바로 접근합니다. 비워 두면 실행 즉시 NullReferenceException이 납니다.

- **spriteRenderer, damageable, meleeDamager** — 좌우 반전·점멸, 체력 관리, 근접 판정
- **facingLeftBulletSpawnPoint, facingRightBulletSpawnPoint, bulletPool** — 총알 생성 위치와 풀
- **cameraFollowTarget** — 카메라가 따라가는 타깃. 바라보는 방향·속도·상하 입력에 따라 localPosition 이 움직인다
- **RandomAudioPlayer 5 개** — footstep / landing / hurt / meleeAttack / rangedAttack

5-3. Chomper 와 Spitter — 같은 컴포넌트, 다른 조종 주체

두 적은 같은 EnemyBehaviour 컴포넌트를 공유합니다. 다른 점은 (가) 인스펙터에서 어떤 필드를 채웠는지와, (나) 누가 EnemyBehaviour의 메서드를 호출하는지뿐입니다. 이 대비는 "상태 기계 방식과 행동 트리 방식이 같은 몸체 위에서 어떻게 달라지는가"를 보여 주는 교재로 쓸 만합니다.

항목	Chomper (근접)	Spitter (원거리)
공유 컴포넌트	EnemyBehaviour	EnemyBehaviour
채우는 필드	meleeDamager, meleeRange, attackDash, attackForce	projectilePrefab, shootingOrigin
조종 주체	Animator에 붙은 SceneLinkedSMB<EnemyBehaviour> 4종	컴포넌트 SpitterBT가 들고 있는 BTAI 행동 트리
상태 표현	Animator 상태(Patrol / RunToTarget / Attack / Death)가 곧 상태	Animator는 모션만 담당. 판단은 트리의 노드가 한다
순찰	ChomperPatrolSMB가 speed만큼 앞을 보고 장애물이 있으면 −speed로 뒤집는다	제자리. 트리가 ScanForPlayer만 반복한다
발견	각 SMB가 ScanForPlayer를 호출 → Spotted 트리거	BT.If(Target == null) 가지 안에서 ScanForPlayer
공격	CheckMeleeAttack이 meleeRange 안을 확인하고 MeleeAttack 트리거	Shooting 트리거 → RememberTargetPos → Attack 상태 대기
장점	Animator 창에서 전이 조건을 눈으로 보고 고칠 수 있다	조건과 순서를 코드 한 곳에서 선언적으로 읽을 수 있다

EnemyBehaviour 의 시야 판정

```
// ScanForPlayer() — 거리 → 각도 두 단계로 걸러낸다
Vector3 dir = PlayerCharacter.PlayerInstance.transform.position - transform.position;
if (dir.sqrMagnitude > viewDistance * viewDistance) return;        // 1) 거리


Vector3 testForward = Quaternion.Euler(0, 0, ... viewDirection) * m_SpriteForward;
float angle = Vector3.Angle(testForward, dir);
if (angle > viewFov * 0.5f) return;                                // 2) 시야각 절반


m_Target = PlayerCharacter.PlayerInstance.transform;
m_TimeSinceLastTargetView = timeBeforeTargetLost;
m_Animator.SetTrigger(m_HashSpottedPara);
```

시야 판정에서 확인된 사실

· viewFov는 시야의 전체 폭(도)이고, 판정에는 그 절반을 쓴다. Walkthrough의 Spitter 예시값 View Fov 182는 좌우 91도씩, 거의 전방 반원을 본다는 뜻이다.

· viewDirection은 스프라이트가 보는 방향을 기준으로 시야 중심을 돌리는 각도다. 코드의 [Range] 제한은 0~360이고 초기값은 0이다.

· ScanForPlayer는 첫 줄에서 PlayerInput.Instance.HaveControl을 확인한다. 씬 전환이나 컷신처럼 플레이어가 조작할 수 없는 동안에는 적이 추격을 시작하지 않는다 — 반응할 수 없는 상대를 때리지 않는다는 배려다.

· CheckTargetStillVisible은 시야 안에 있으면 타이머를 timeBeforeTargetLost로 되돌리고, 0 이하가 되면 ForgetTarget으로 TargetLost 트리거를 건다.

Spitter 의 탄도 계산

Spitter는 플레이어를 향해 직선으로 쏘지 않습니다. 중력을 고려한 포물선 해를 풀어 발사 속도 벡터를 구합니다. 이때 쓰는 발사 속력은 인스펙터 값이 아니라 함수 안에 박힌 상수입니다.

```
// EnemyBehaviour.GetProjectilVelocity(target, origin)
const float projectileSpeed = 30.0f;          // ← 인스펙터 노출 없음


Vector3 toTarget = target - origin;
float gSquared   = Physics.gravity.sqrMagnitude;
float b        = projectileSpeed * projectileSpeed + Vector3.Dot(toTarget, Physics.gravity);
float discriminant = b * b - gSquared * toTarget.sqrMagnitude;
// discriminant < 0 이면 최대 속력으로도 닿지 않는 거리 → 대체 처리
```

발사 지점은 shootingOrigin의 로컬 좌표이고, 스프라이트가 뒤집혀 있으면 x를 반전해 반대쪽에서 나가게 합니다. 목표 지점은 실시간 플레이어 위치가 아니라 RememberTargetPos가 기록해 둔 m_TargetShootPosition입니다. 즉 Spitter는 "조준한 순간의 위치"를 향해 쏘므로, 플레이어가 움직이면 피할 수 있습니다.

5-4. Gunner (MissileGolem) — 보스 설계

보스는 MissileGolem 하나가 BTAI 행동 트리로 전 과정을 선언합니다. 라운드 수는 rounds 배열의 길이가 정하고, 각 라운드는 자신의 체력·쉴드 체력·플랫폼 속도를 따로 가집니다.

필드	의미	코드 기본값
rounds[]	라운드 정의 배열. 길이가 곧 라운드 수	(초기화 없음)
rounds[].bossHP	그 라운드의 본체 체력	10
rounds[].shieldHP	그 라운드의 쉴드 체력	10
rounds[].platformSpeed	그 라운드에 활성화되는 MovingPlatform들의 speed	1
rounds[].platforms[]	그 라운드에 활성화할 이동 플랫폼	(초기화 없음)
rounds[].enableOnProgress[]	그 라운드에 함께 켤 오브젝트	(초기화 없음)
laserStrikeCount	빔 패턴 한 번에 반복할 발사 횟수	2
laserTrackingSpeed	빔이 플레이어를 따라 도는 각속도 (도/초)	30
delay	패턴 사이 기본 대기 시간 (초)	2
lightningTime	번개가 지속되는 시간 (초)	1
beamDelay / grenadeDelay / lightningDelay	각 패턴의 모션과 실제 발사 사이 지연	(초기화 없음)
cleanupDelay / deathDelay	격파 후 정리·종료 연출 간격	(초기화 없음)

보스 행동 트리 (뼈대)

```
ai = BT.Root().OpenBranch(
  BT.SetActive(beamLaser, false),
  BT.Repeat(rounds.Length).OpenBranch(          // 라운드 수만큼 반복
    BT.Call(NextRound),                         // 체력 설정 + 무적 ON + 플랫폼 활성화
    BT.If(GrenadeEnabled).OpenBranch(BT.Trigger(animator, "Enabled")),
    BT.Wait(delay), BT.Call(ActivateShield), BT.Wait(delay),
    BT.While(ShieldIsUp).OpenBranch(            // 쉴드가 깨질 때까지
      BT.RandomSequence(new int[] { 1, 6, 4, 4 }).OpenBranch(
        /*1*/ 걷기, /*6*/ 빔, /*4*/ 번개, /*4*/ 수류탄 )
    ),
    /* 빔 끄기 + Grenade·Beam·Lightning 트리거 리셋 + "Disabled" */
    BT.While(IsAlive).OpenBranch(BT.Wait(0))    // 본체 체력 0까지 대기
  ),
  BT.Trigger(animator, "Death"), /* Cleanup → Die */, BT.Terminate()
);
```

패턴 추첨의 가중치

RandomSequence는 자식 노드마다 가중치를 받아 누적 가중치로 추첨합니다. 보스의 가중치는 { 1, 6, 4, 4 } 이고 합이 15이므로, 각 패턴이 뽑힐 확률은 다음과 같습니다.

가중치	패턴	확률	조건
1	걷기 (Walk → Idle 대기)	1/15 ≈ 6.7%	없음
6	빔 (laserStrikeCount회 반복 발사)	6/15 = 40%	없음
4	번개	4/15 ≈ 26.7%	BT.If(EllenOnFloor) — 플레이어가 바닥에 있을 때만
4	수류탄	4/15 ≈ 26.7%	BT.If(GrenadeEnabled) — 2라운드 이상에서만

보스 설계에서 읽을 수 있는 의도

· 빔에 가중치 6을 준 것은 빔이 이 보스의 "주 공격"이라는 선언이다. 걷기에 1만 준 것은 걷기가 공격이 아니라 숨 고르기용 여백이라는 뜻이다.

· 빔 조준은 Quaternion.RotateTowards로 laserTrackingSpeed만큼만 돌린다. 즉시 조준이 아니라 "느리게 따라오는 조준"이라 플레이어에게 회피 여지가 남는다.

· 번개는 플레이어가 바닥에 있을 때만, 수류탄은 2라운드 이상에서만 나온다. 조건부 패턴으로 라운드가 올라갈수록 선택지가 늘어나는 구조다.

· 본체 체력은 라운드마다 SetHealth로 다시 채워지므로, 체력 슬라이더는 Damageable의 현재 체력을 그대로 쓰지 못한다. MissileGolem이 모든 라운드의 bossHP를 합산한 m_TotalHealth를 따로 들고 Damaged 콜백에서 직접 깎는다.

6. 레벨과 환경 (Levels & Environment)

6-1. 타일맵과 타일셋

레벨 지형은 Unity의 Tilemap으로 그립니다. Window > 2D > Tile Palette를 열고 Tileset_Gamekit을 선택하면 두 가지 타일셋을 쓸 수 있습니다.

- **Grassy Rocks** — 풀과 바위로 이루어진 지상 지형. 게임 초반의 야외 구역, Ellen 이 불시착한 행성 표면.
- **Alien Structure** — 고대 외계 문명이 남긴 인공 구조물. 지하 묘소와 폐허 내부, 후반 구역.

타일은 지형만이 아니다

· 타일에 AudioSurface 컴포넌트를 붙여 TileBase를 지정하면, RandomAudioPlayer가 그 타일 위에서 다른 효과음을 쓴다. 풀 위를 걸을 때와 금속 위를 걸을 때 발소리가 달라지는 것이 이 구조다.

· PlayerCharacter.FindCurrentSurface()가 접지 콜라이더에서 밟고 있는 TileBase를 찾아 m_CurrentSurface에 넣고, PlayFootstep과 착지음이 그 값을 인수로 넘긴다.

· VFXController도 같은 TileBase를 받아 먼지 이펙트 프리팹을 타일별로 교체한다. 소리와 이펙트가 같은 한 가지 정보(지금 밟고 있는 타일)에서 갈라져 나온다.

6-2. 상호작용 오브젝트

아래는 Prefabs 폴더에서 끌어다 놓아 쓰는 주요 오브젝트와, 그 뒤에 붙어 있는 컴포넌트입니다. 필드의 구체적인 기본값은 8-3절의 수치 표에 모아 두었습니다.

프리팹 / 오브젝트	역할	핵심 컴포넌트	인스펙터 주요 필드
MovingPlatform	정해진 경로를 왕복·순환하는 이동 바닥	MovingPlatform PlatformCatcher Rigidbody2D	speed, platformType, localNodes[], waitTimes[], isMovingAtStart, startMovingOnlyWhenVisible
PressurePad	물건이나 플레이어가 올라가면 이벤트를 발생시키는 압력판	PressurePad PlatformCatcher	activationType, requiredCount는 requiredMass, boxes[], OnPressed / OnRelease
Door	압력판·열쇠와 묶여 열리는 문	Animator	PressurePad의 On Pressed에서 Animator.Play("DoorOpening"로 연다
HubDoor	열쇠를 모두 모아야 열리는 거점 문	HubDoor SpriteRenderer	requiredInventoryItemKeys[], unlockStateSprites[] (키 개수와 길이 대응), characterInventory, onUnlocke
PushableBox	밀 수 있는 상자. Damager를 추가하면 적을 죽이는 무기가 된다	Pushable Rigidbody2D (+ Damager 선택)	playerPushingLeft/RightPositio pushablePosition, Damager.hittableLayers = Ene
HealthPickup	체력 회복 아이템	HealthPickup	healthAmount, OnGivingHealt

프리팹 / 오브젝트	역할	핵심 컴포넌트	인스펙터 주요 필드
WeaponPickup	근접·원거리 무기 획득 지점	WeaponPickup InteractOnTrigger2D	spriteRenderer, blueMotes, interactOnTrigger2D, dataSettings (획득 여부 영속화
InventoryItem (열쇠)	주우면 인벤토리에 키 문자열이 들어가는 아이템	InventoryItem CircleCollider2D	inventoryKey (대소문자 구분), layers, disableOnEnter, clip
EnemySpawner	적을 일정 간격으로 내보내는 소환 지점	EnemySpawner (ObjectPool 상속)	prefab, initialPoolCount, totalEnemiesToBeSpawned, concurrentEnemiesToBeSpawn spawnArea, spawnDelay, removalDelay
Checkpoint	리스폰 지점	Checkpoint	respawnFacingLeft
WaterArea	물결치는 수면. 같은 오브젝트의 Damager가 피해를 준다	WaterArea MeshRenderer / MeshFilter BoxCollider2D (+ Damager)	size, offset, pointPerUnits, dampening, tension, neighbourTransfer, sortingLayerOrder
TransitionPoint	다른 구역·씬으로 넘어가는 지점	TransitionPoint Collider2D	transitioningGameObject, transitionType, newSceneName, transitionDestinationTag, transitionWhen, requiresInventoryCheck
SceneTransitionDestination	넘어온 플레이어가 놓이는 도착 지점	SceneTransitionDestination	destinationTag (A~G), transitioningGameObject, OnReachDestination
InteractOnTrigger2D / InteractOnButton2D	영역 진입 또는 E 입력으로 이벤트를 발생시키는 범용 상호작용	InteractOnTrigger2D InteractOnButton2D	layers, OnEnter / OnExit, inventoryChecks[], OnButtonPress (Button 쪽만)
DirectorTrigger	Timeline(PlayableDirector)을 재생시키는 트리거	DirectorTrigger	triggeringGameObject, directo triggerType, OnDirectorPlay / OnDirectorFinish

6-3. 이동 플랫폼의 경로 처리

MovingPlatform은 localNodes 배열의 노드를 월드 좌표로 바꿔 두고(m_WorldNode), 매 프레임 speed × Time.deltaTime만큼의 거리를 소비하며 다음 노드로 나아갑니다. 남은 거리가 다음 노드까지의 거리보다 크면 while 루프를 한 번 더 돌아 그 프레임 안에서 노드를 지나칩니다. 즉 speed가 매우 크더라도 노드를 건너뛰지 않습니다.

platformType	끝에 도달했을 때	시작으로 되돌아왔을 때
BACK_FORTH	m_Next를 길이−2로, 방향을 −1로 (왕복)	m_Next를 1로, 방향을 +1로
LOOP	m_Next를 0으로 (처음으로 순환)	m_Next를 길이−1로
ONCE	m_Next를 되돌리고 StopMoving()	m_Next를 되돌리고 StopMoving()

노드에 도달하면 waitTimes[m_Current]를 m_WaitTime에 넣고, 그 값이 0.001보다 크면 루프를 즉시 빠져나옵니다. 플랫폼은 그 자리에서 대기합니다. 실제 이동은 transform을 직접 옮기지 않고 m_Rigidbody2D.MovePosition으로 하고, 같은 변위를 platformCatcher.MoveCaughtObjects에 그대로 넘깁니다.

7. 시스템 설계 (System Design)

이 장은 키트의 코드가 어떻게 조직되어 있는지를 열 개의 소절로 나누어 기술합니다. 각 소절은 (가) 무엇을 해결하려는 구조인지, (나) 실제 호출 흐름, (다) 확인된 수치와 식별자의 순서로 씁니다.

7-1. 입력 — PlayerInput 의 2 단계 차단 구조

입력은 두 가지 서로 다른 이유로 막힐 수 있습니다. 하나는 "아직 그 능력을 얻지 못했다"이고, 다른 하나는 "지금은 조작할 수 없는 상황이다"입니다. InputComponent는 이 둘을 별도의 플래그로 구분합니다.

구분	플래그	켜고 끄는 메서드	막혔을 때의 동작	쓰이는 상황
능력 차단	m_Enabled (SerializeField)	Enable() / Disable()	Get()이 Down·Held·Up을 전부 false로 덮고 반환. 축은 Value를 0으로	무기를 얻기 전의 MeleeAttack·RangedAttack. F12 디버그 메뉴가 토글하는 것도 이 플래그다

구분	플래그	켜고 끄는 메서드	막혔을 때의 동작	쓰이는 상황
상황 차단	m_GettingInput	GainControl() / ReleaseControl(resetValues)	Get()이 아무것도 하지 않고 반환해 직전 값이 그대로 남는다. resetValues가 true면 먼저 값을 정리한다	씬 전환, 컷신, 일시정지. SceneController와 PlayerCharacter가 호출한다

입력 처리에서 확인된 두 가지 설계

· FixedUpdate 사이에 눌렸다 떼어진 입력을 놓치지 않는 장치 — Update()는 m_FixedUpdateHappened를 보고, 아직 FixedUpdate가 지나지 않았으면 이전 프레임의 Down·Held·Up을 OR로 누적해 유지한다. 이동이 FixedUpdate에서 일어나므로 이 보정이 없으면 한 프레임짜리 점프 입력이 사라질 수 있다.

· 일시정지 중에도 입력이 살아 있는 장치 — Update()는 Mathf.Approximately(Time.timeScale, 0)이면 fixedUpdateHappened를 true로 간주한다. timeScale이 0이면 FixedUpdate가 돌지 않으므로, 이 예외가 없으면 일시정지 메뉴에서 버튼이 먹지 않는다.

· 일시정지 진입은 ReleaseControl(false)로 전부 막은 다음 Pause.GainControl()만 되돌린다. 해제 버튼만 살려 두는 방식이다.

```
// PlayerCharacter.Update() — 일시정지 진입
if (PlayerInput.Instance.Pause.Down && !m_InPause)
{
    if (ScreenFader.IsFading) return;
    PlayerInput.Instance.ReleaseControl(false);   // 값은 유지하고 입력만 차단
    PlayerInput.Instance.Pause.GainControl();     // Pause 만 되살린다
    m_InPause = true;
    Time.timeScale = 0;
    SceneManager.LoadSceneAsync("UIMenus", LoadSceneMode.Additive);
}
```

7-2. 캐릭터 이동과 지면 판정

이동은 CharacterController2D가 담당합니다. PlayerCharacter와 EnemyBehaviour가 Move(movement)로 이번 프레임의 변위를 쌓아 두면, FixedUpdate가 그것을 Rigidbody2D.MovePosition으로 한 번에 적용하고, 곧바로 접지와 천장을 판정합니다.

```
// CharacterController2D.FixedUpdate()
m_PreviousPosition = m_Rigidbody2D.position;
m_CurrentPosition  = m_PreviousPosition + m_NextMovement;
Velocity           = (m_CurrentPosition - m_PreviousPosition) / Time.deltaTime;


m_Rigidbody2D.MovePosition(m_CurrentPosition);
m_NextMovement = Vector2.zero;


CheckCapsuleEndCollisions();        // 아래쪽 – 접지 판정
CheckCapsuleEndCollisions(false);   // 위쪽  – 천장 판정
```

아래쪽 레이캐스트 3 발

접지 판정은 캡슐 콜라이더의 아래쪽에서 좌·중앙·우 세 지점에서 Physics2D.Raycast를 쏘는 방식입니다. 세 발을 쓰는 이유는 경사면과 플랫폼 경계에서 한 발만으로는 접지 여부가 깜빡이기 때문입니다.

항목	값 · 계산식
시작점	rigidbody.position + capsule.offset + down × (size.y × 0.5 − size.x × 0.5) 를 중심으로, 좌: 중심 + left × size.x × 0.5 / 중앙: 중심 / 우: 중심 + right × size.x × 0.5
방향	Vector2.down (아래쪽) / Vector2.up (위쪽 천장 판정)
사거리	m_Capsule.size.x * 0.5f + groundedRaycastDistance * 2f
필터	groundedLayerMask만, 트리거 제외 (useTriggers = false)
법선 합산	맞은 레이의 normal을 모두 더한 뒤 Normalize. 하나도 맞지 않으면 IsGrounded = false
최종 판정	IsGrounded = relativeVelocity.y <= 0f — 올라가는 중이면 접지로 보지 않는다
중앙 보정	중앙 레이가 맞았다면, 그 교점 높이가 캡슐 바닥 + groundedRaycastDistance 아래일 때만 접지로 인정한다

relativeVelocity가 왜 상대 속도인가

· IsGrounded 판정에 쓰는 relativeVelocity는 컨트롤러 자신의 Velocity에서, 밟고 있는 MovingPlatform의 Velocity / Time.deltaTime을 뺀 값이다.

· 이 보정이 없으면 위로 올라가는 플랫폼에 서 있을 때 Velocity.y가 양수가 되어 "올라가는 중이니 접지가 아니다"로 판정되고, 캐릭터가 플랫폼 위에서 계속 공중 상태로 떨린다.

· PlayerCharacter에도 같은 목적의 상수가 있다 — k_GroundedStickingVelocityMultiplier = 3f. 주석은 "수직으로 움직이는 플랫폼에 캐릭터가 붙어 있도록 돕기 위한 값"이라고 적혀 있다.

7-3. 피해 시스템 — Damager / Damageable

피해는 콜라이더 충돌 메시지(OnCollisionEnter2D 등)로 받지 않습니다. Damager가 FixedUpdate마다 자기 판정 영역을 능동적으로 조회하는 방식입니다. 이 선택이 가지는 결과가 몇 가지 있습니다.

- **공격 영역을 애니메이션과 독립적으로 켜고 끌 수 있다:** EnableDamage() / DisableDamage()로 m_CanDamage 만 바꾸면 FixedUpdate 가 조회를 건너뛴다. 근접 공격 모션의 특정 구간에서만 판정을 켜는 처리가 Animator 이벤트 한 줄로 끝난다.
- **맞는 쪽이 Rigidbody 를 가질 필요가 없다:** Physics2D.OverlapArea 는 콜라이더만 찾는다. 정적인 함정이 움직이는 플레이어를 때리는 것도, 그 반대도 같은 코드로 처리된다.
- **한 프레임에 여러 대상을 동시에 때린다:** 조회 결과 배열(m_AttackOverlapResults, 길이 10)에 들어온 모든 대상에 대해 순서대로 TakeDamage 를 호출한다. 11 번째부터는 그 프레임에 맞지 않는다.
- **판정 영역이 스프라이트 방향을 따라간다:** offsetBasedOnSpriteFacing 이 켜져 있고 spriteRenderer.flipX 가 원래와 달라지면 offset.x 의 부호가 뒤집힌다. 뒤돌아서 공격해도 앞쪽을 때린다.

Damageable 쪽에는 네 개의 UnityEvent가 있습니다. OnHealthSet(체력이 바뀔 때마다 — UI 갱신용), OnTakeDamage(피해를 받았을 때), OnDie(체력이 0 이하가 되었을 때), OnGainHealth(회복했을 때)입니다. 캐릭터의 반응은 전부 이 네 이벤트에 인스펙터로 연결됩니다.

7-4. 애니메이터 연동 — SceneLinkedSMB

Unity의 기본 StateMachineBehaviour에는 두 가지 불편이 있습니다. 첫째, 자신이 붙어 있는 오브젝트의 컴포넌트를 쓰려면 매번 GetComponent를 호출해야 합니다. 둘째, OnStateEnter·OnStateUpdate·OnStateExit 세 콜백만으로는 "전이 중"과 "전이가 끝난 뒤"를 구분할 수 없습니다. SceneLinkedSMB는 두 문제를 함께 해결합니다.

참조 주입

```
// 씬의 MonoBehaviour 가 Start() 에서 한 번 호출한다
SceneLinkedSMB<PlayerCharacter>.Initialise(animator, this);


// 내부
var smbs = animator.GetBehaviours<SceneLinkedSMB<TMonoBehaviour>>();
for (int i = 0; i < smbs.Length; i++)
    smbs[i].InternalInitialise(animator, monoBehaviour);   // 저장 + OnStart
```

이후 각 SMB는 m_MonoBehaviour로 대상 컴포넌트를 바로 씁니다. ChomperPatrolSMB가 m_MonoBehaviour.CheckForObstacle(dist)을 부르는 것이 그 예입니다.

7 단계 콜백

SceneLinkedSMB는 StateMachineBehaviour의 세 콜백을 sealed로 닫고(SealedSMB), 대신 아래 일곱 개를 제공합니다. 구분 기준은 animator.IsInTransition(layerIndex)과 내부 플래그 m_FirstFrameHappened · m_LastFrameHappened입니다.

콜백	불리는 시점
OnStart	Initialise가 호출된 직후 한 번. 참조를 받아 초기화할 곳
OnSLStateEnter	이 상태로의 전이가 시작될 때 (업데이트보다 먼저)
OnSLTransitionToStateUpdate	이 상태로 전이하는 동안 매 프레임
OnSLStatePostEnter	이 상태로의 전이가 끝난 첫 프레임
OnSLStateNoTransitionUpdate	전이 중이 아닐 때 매 프레임 — 실제 게임플레이 로직이 들어가는 곳
OnSLStatePreExit	이 상태에서 빠져나가는 전이가 시작된 첫 프레임. 전이 시간이 한 프레임보다 짧으면 불리지 않는다
OnSLStateExit	이 상태에서 빠져나가는 전이가 끝난 뒤

각 콜백은 AnimatorControllerPlayable을 받는 오버로드도 함께 제공되며, 두 버전이 연달아 호출됩니다. Ellen의 SMB는 19개(Player 15개, Enemies 4개)이고, 대부분이 OnSLStateNoTransitionUpdate 하나만 재정의합니다.

7-5. 적 AI — 두 가지 방식

(가) 상태 기계 방식 — Chomper

Animator의 상태가 곧 적의 상태입니다. 각 상태에 붙은 SMB가 매 프레임 EnemyBehaviour의 메서드를 호출하고, 상태 전이는 Animator 창의 전이 조건이 결정합니다. EnemyBehaviour가 쓰는 Animator 파라미터는 Spotted · Shooting · TargetLost · MeleeAttack · Hit · Death · Grounded 일곱 개입니다.

```
// ChomperPatrolSMB — 순찰 상태에서 매 프레임
float dist = m_MonoBehaviour.speed;
if (m_MonoBehaviour.CheckForObstacle(dist))
{
    m_MonoBehaviour.SetHorizontalSpeed(-dist);   // 이동 벡터를 뒤집으면
    m_MonoBehaviour.UpdateFacing();        // 스프라이트와 forward 도 함께 뒤집힌다
}
else
    m_MonoBehaviour.SetHorizontalSpeed(dist);


m_MonoBehaviour.ScanForPlayer();
```

장애물 판정을 EnemyBehaviour가 아니라 각 SMB에서 하는 이유는 소스 주석에 적혀 있습니다 — 상태마다 장애물에 다르게 반응하고 싶기 때문입니다. ChomperRunToTargetSMB는 장애물을 만나면 방향을 바꾸는 대신 추격 자체를 포기합니다.

(나) 행동 트리 방식 — Spitter 와 Gunner

BTAI 네임스페이스의 행동 트리는 매 프레임 루트 노드를 Tick()하고, 각 노드가 Success · Failure · Continue · Abort 중 하나를 반환합니다. Abort는 공식 Advanced Topics 문서에 나오지 않는 네 번째 값으로, Terminate 노드가 트리를 영구 종료시킬 때 쓰입니다.

노드	종류	동작 (코드 확인)
Sequence	분기	자식이 Success면 다음으로, 끝까지 가면 Success. Failure면 activeChild를 0으로 되돌리고 Failure
Selector	분기	자식이 Success면 즉시 Success. Failure면 다음 자식으로, 끝까지 실패하면 Failure
Block	분기 (추상)	자식이 Continue가 아니면 다음으로 넘어간다 — 즉 Failure와 Abort를 무시한다. 끝까지 돌면 Success. Root · Repeat · While · ConditionalBranch · RandomSequence의 공통 부모
Root	분기	Block을 while 루프로 감싸 Continue가 나올 때까지 여러 자식을 한 프레임에 연달아 처리. 자식이 Abort를 올리면 isTerminated를 세우고 이후 항상 Abort
Repeat(n)	분기	n회를 채우면 Success를 내고 currentCount를 0으로 되돌린다(재사용 가능). 그 전에는 Continue
RandomSequence(w[])	분기	누적 가중치 배열(m_AddedWeight)로 자식 하나를 추첨한다. 가중치가 모자라면 그 뒤는 모두 1. 자식이 끝나면 다시 추첨

노드	종류	동작 (코드 확인)
While(fn)	분기	조건이 true인 동안 자식을 돌리고 항상 Continue를 반환한다. false가 되면 ResetChildren() 후 Failure. Success를 내는 경로가 없다
ConditionalBranch(fn) = If	분기	조건이 true이면 자식 블록을 돌리고, false이면 Failure
Condition(fn)	말단	fn()의 결과를 Success / Failure로 그대로 돌려준다
Call(fn) / RunCoroutine(fn)	말단	System.Action을 한 번 호출하고 Success / IEnumerator<BTState>를 돌려 Continue인 동안 이어 간다
Wait(sec)	말단	Time.time 기준으로 sec초를 센다. 그동안 Continue, 지나면 Success (future를 −1로 리셋해 재사용)
Trigger / SetBool / SetActive	말단	각각 SetTrigger(set이 false면 ResetTrigger) · bool 파라미터 설정 · GameObject.SetActive를 하고 Success
WaitForAnimatorState / WaitForAnimatorSignal	말단	지정한 상태에 들어갈 때까지 / 그 상태에서 신호가 올 때까지 Continue
Log(msg) / Terminate	말단	Debug.Log 후 Success / Abort를 반환해 상위 Root를 영구 종료시킨다

```
// SpitterBT.OnEnable() — 트리 전체가 이 한 덩어리다
m_Ai.OpenBranch(
    BT.If(() => { return m_EnemyBehaviour.Target != null; }).OpenBranch(
        BT.Call(m_EnemyBehaviour.CheckTargetStillVisible),
        BT.Call(m_EnemyBehaviour.OrientToTarget),
        BT.Trigger(m_Animator, "Shooting"),
        BT.Call(m_EnemyBehaviour.RememberTargetPos),
        BT.WaitForAnimatorState(m_Animator, "Attack")
    ),
    BT.If(() => { return m_EnemyBehaviour.Target == null; }).OpenBranch(
        BT.Call(m_EnemyBehaviour.ScanForPlayer)
    )
);

private void Update() { m_Ai.Tick(); }
```

7-6. 오브젝트 풀

총알·적·이펙트는 필요할 때 Instantiate하지 않고, 미리 만들어 둔 것을 꺼내 쓰고 되돌려 놓습니다.

Core/ObjectPool.cs에 네 개의 제네릭 클래스가 짝을 이루어 들어 있습니다.

클래스	상속	성격	역할
ObjectPool<TPool, TObject>	—	MonoBehaviour	풀 본체. prefab, initialPoolCount(기본 10), pool 리스트를 들고 Pop / Push를 제공
ObjectPool<TPool, TObject, TInfo>	위를 상속	MonoBehaviour	Pop(TInfo info) 추가. 꺼낼 때 정보를 함께 넘길 수 있다 (예: 생성 위치 Vector2)
PoolObject<TPool, TObject>	—	[Serializable] MonoBehaviour 아님	풀 항목. inPool · instance · objectPool을 들고 SetReferences / WakeUp / Sleep / ReturnToPool을 제공
PoolObject<TPool, TObject, TInfo>	위를 상속	[Serializable]	WakeUp(TInfo info) 추가

수명 순서

```
Start()  →  CreateNewPoolObject() 를 initialPoolCount 번
  ├ new TObject() ;  instance = Instantiate(prefab) ;  SetParent(this)
  ├ inPool = true
  ├ SetReferences(this)   – 항목이 instance 에서 컴포넌트를 캐시하는 곳
  └ Sleep()               – 보통 SetActive(false)


(사용자) Pop() / Pop(info)   inPool == true 인 첫 항목을 찾아
  └ inPool = false ;  WakeUp() / WakeUp(info)   – 위치를 놓고 SetActive(true)


(항목 자신) ReturnToPool()   일이 끝나면 스스로 돌아온다
  └ objectPool.Push(this)  →  inPool = true ;  Sleep()
```

풀은 줄어들지 않는다

· Pop()은 pool 리스트에서 항목을 제거하지 않는다. inPool 플래그만 false로 바꾸고 같은 항목을 돌려준다. 반납도 플래그를 true로 되돌리는 것뿐이다.

· 모든 항목이 사용 중이면 CreateNewPoolObject()로 새로 만들어 pool에 추가한다. 따라서 풀 크기는 그 씬에서의 최대 동시 사용량까지 커지고, 그 뒤로 줄어들지 않는다.

· 이는 의도된 설계다 — 메모리를 조금 더 쓰는 대신 런타임 할당을 없앤다. 다만 적 소환기를 과하게 설정하면 풀이 계속 커지므로, initialPoolCount를 예상 최대치에 맞추어 두는 편이 좋다.

· 풀을 쓰는 구현은 BulletPool / BulletObject, EnemySpawner / Enemy, VFXInstancePool / VFXInstance 세 쌍이다.

7-7. 상호작용과 인벤토리

상호작용은 두 계열로 나뉩니다. 영역에 들어가면 바로 발생하는 것(InteractOnTrigger2D)과, 영역 안에서 E

를 눌러야 발생하는 것(InteractOnButton2D)입니다. 후자는 전자를 상속하면서 ExecuteOnEnter·ExecuteOnExit를 재정의해 "지금 버튼을 받을 수 있다"는 플래그만 세우고, 실제 발생은 Update에서 Interact.Down을 확인해 처리합니다.

```
// InteractOnButton2D.Update()
if (m_CanExecuteButtons)
{
    if (OnButtonPress.GetPersistentEventCount() > 0 && PlayerInput.Instance.Interact.Down)
        OnButtonPress.Invoke();
}
```

GetPersistentEventCount() 확인이 들어 있어, 인스펙터에 아무것도 연결하지 않은 상호작용 지점은 E를 눌러도 반응하지 않습니다.

인벤토리 전달 경로

열쇠를 주워 UI에 아이콘이 켜지기까지의 경로입니다. 중간의 모든 연결이 UnityEvent이거나 인스펙터 참조라는 점이 이 키트의 성격을 잘 보여 줍니다.

```
InventoryItem.OnTriggerEnter2D(other)
  │   layers.Contains(other) 확인
  └→ other.GetComponent<InventoryController>().AddItem(inventoryKey)
        │
        ├ m_InventoryItems (HashSet<string>) 에 추가
        └ key 가 일치하는 InventoryEvent 의 OnAdd.Invoke()
              │   ↑ 인스펙터에서 연결
              └→ PlayerCharacter.KeyInventoryEvent()
                    └→ KeyUI.Instance.ChangeKeyUI(m_InventoryController)
                          └ keyNames[i] 마다
                            m_KeyIconAnimators[i].SetBool("Active", HasItem(keyNames[i]))
```

키 문자열에 관한 주의

· 키는 string이고 HashSet<string>.Contains로 비교하므로 대소문자를 구분한다. InventoryItem의 inventoryKey, InventoryController의 InventoryEvent.key, KeyUI의 keyNames[], HubDoor의 requiredInventoryItemKeys[], TransitionPoint의 inventoryCheck에 적는 문자열이 모두 정확히 같아야 한다.

· 한 글자만 달라도 아무 오류 없이 조용히 동작하지 않는다. 컴파일 시점에 잡히지 않는 종류의 실수이므로, 새 열쇠를 추가할 때 가장 먼저 점검할 항목이다.

· InventoryController에는 [ContextMenu("Dump")] Dump() 함수가 있다. 플레이 중 인스펙터의 컨텍스트 메뉴에서 호출하면 현재 가진 키를 모두 콘솔에 찍어 주므로 디버깅에 쓸 수 있다.

· InventoryChecker는 inventoryItems[] 전부를 가지고 있으면 OnHasItem, 하나라도 없으면 OnDoesNotHaveItem을 발생시킨다. 단, 없는 항목을 발견하는 즉시 반환하므로 OnDoesNotHaveItem은 한 번만 불린다.

HubDoor는 조금 다릅니다. requiredInventoryItemKeys를 순회하며 가진 키를 세서 stateIndex를 만들고, unlockStateSprites[stateIndex]로 문의 스프라이트를 바꿉니다. 그래서 두 배열의 길이가 대응해야 합니다 — 키가 N개면 unlockStateSprites의 마지막 인덱스가 N−1입니다. stateIndex가 requiredInventoryItemKeys.Length − 1에 도달하면 onUnlocked가 발생합니다.

7-8. 씬 전환과 데이터 영속화

구역을 옮길 때 해야 하는 일이 많습니다. 현재 상태를 저장하고, 입력을 막고, 화면을 어둡게 하고, 새 씬을 비동기로 읽고, 저장한 상태를 복원하고, 플레이어를 도착 지점에 놓고, 다시 밝게 하고, 조작을 돌려주어야 합니다. SceneController.Transition 코루틴이 이 순서를 한곳에 모아 두었습니다.

순서	하는 일	호출
①	전환 중 표시를 세운다	m_Transitioning = true
②	현재 씬의 모든 영속화 대상을 저장한다	PersistentDataManager.SaveAllData()
③	플레이어 입력을 차단한다	m_PlayerInput.ReleaseControl()
④	화면을 어둡게 한다 (로딩 화면)	ScreenFader.FadeSceneOut(FadeType.Loading)
⑤	등록된 영속화 대상 목록을 비운다 (저장된 데이터는 남는다)	PersistentDataManager.ClearPersisters()
⑥	새 씬을 비동기로 읽는다	SceneManager.LoadSceneAsync(newSceneName)
⑦	새 씬의 PlayerInput을 찾아 다시 차단한다	FindFirstObjectByType → ReleaseControl()
⑧	저장한 데이터의 복원을 예약한다	PersistentDataManager.LoadAllData()
⑨	도착 지점을 태그로 찾는다	GetDestination(destinationTag)
⑩	전환 대상(보통 플레이어)을 도착 지점에 놓는다	SetEnteringGameObjectLocation(entrance)
⑪	구역 전환이면 재시작 지점을 갱신한다	SetupNewScene → SetZoneStart
⑫	도착 이벤트를 발생시킨다	entrance.OnReachDestination.Invoke()
⑬	화면을 밝게 한다	ScreenFader.FadeSceneIn()
⑭	조작을 돌려주고 전환 중 표시를 내린다	GainControl() / m_Transitioning = false

이 순서에서 중요한 두 지점

· 저장이 페이드보다 먼저다 (② → ④). 화면을 어둡게 한 뒤에 저장하면 그 사이에 벌어진 변화가 반영되지 않을 수 있다. 저장을 가장 먼저 끝내 두는 것이 안전하다.

· LoadAllData는 즉시 복원하지 않는다 (⑧). PersistentDataManager.LoadAllDataInternal은 복원 작업을 schedule 델리게이트에 넣어 두고, 자신의 Update()에서 꺼내 실행한다. 새 씬의 오브젝트들이 OnEnable에서 RegisterPersister를 끝낼 시간을 주기 위한 한 프레임 지연이다.

· ClearPersisters(⑤)는 등록 목록(m_DataPersisters)만 비우고 저장소(m_Store)는 건드리지 않는다. 그래서 다음 씬에서 같은 dataTag를 가진 오브젝트가 등록되면 이전 값을 그대로 받아 간다.

IDataPersister

영속화에 참여하려면 IDataPersister 인터페이스를 구현하고, 직렬화된 DataSettings 필드를 하나 가지면 됩니다. OnEnable에서 RegisterPersister, OnDisable에서 UnregisterPersister를 부르는 것이 관례입니다.

```
public interface IDataPersister
{
    DataSettings GetDataSettings();
    void SetDataSettings(string dataTag, DataSettings.PersistenceType persistenceType);
    Data SaveData();
    void LoadData(Data data);
}


// DataSettings
public string dataTag = System.Guid.NewGuid().ToString();   // 기본값: 새 GUID
public PersistenceType persistenceType = PersistenceType.ReadWrite;
```

PersistenceType	의미
DoNotPersist	저장도 복원도 하지 않는다
ReadOnly	복원만 한다. 저장은 하지 않으므로 처음 설정한 값이 유지된다
WriteOnly	저장만 한다. 복원하지 않으므로 씬에 들어올 때마다 초기 상태로 돌아간다
ReadWrite	저장과 복원을 모두 한다 (기본값)

dataTag의 기본값이 새 GUID라는 점이 중요합니다. 같은 오브젝트가 씬을 넘나들며 자기 상태를 유지하려면 두 씬에서 같은 dataTag를 써야 하는데, 기본값에 맡기면 서로 다른 GUID가 되어 데이터가 이어지지 않습니다. Walkthrough에서 Data Tag를 직접 입력하라고 안내하는 이유입니다.

IDataPersister 를 구현한 8 개 클래스

클래스	저장하는 것	Data 형태
PlayerInput	근접·원거리 공격이 허용된 상태인지	Data<bool, bool>
Damageable	현재 체력과, 씬 재진입 때 체력을 초기화할지 여부	Data<int, bool>
EnemySpawner	지금까지 소환한 적의 누적 수	Data<int>
WeaponPickup	무기를 이미 집었는지	(WeaponPickup.cs 참조)
HubDoor	현재 표시 중인 문 스프라이트	Data<Sprite>
InventoryItem	아이템이 이미 수집되었는지	(InventoryItem.cs 참조)
InventoryController	가지고 있는 키 문자열 집합	Data<HashSet<string>>
DirectorTrigger	Timeline이 이미 재생되었는지	(DirectorTrigger.cs 참조)

Damageable의 LoadData가 Data<int, bool>의 두 값을 쓰는 방식이 흥미롭습니다. value1(m_ResetHealthOnSceneReload)이 true면 저장된 체력을 무시하고 startingHealth로 되돌립니다. 즉 "한 번 죽은 적은 다시 들어왔을 때 체력이 가득 찬 상태로 부활한다"는 규칙이 이 한 줄에 들어 있습니다.

```
// Damageable.LoadData(Data data)
Data<int, bool> healthData = (Data<int, bool>)data;
m_CurrentHealth = healthData.value1 ? startingHealth : healthData.value0;
OnHealthSet.Invoke(this);
```

7-9. 소리와 이펙트

RandomAudioPlayer — 타일별 소리 오버라이드

같은 동작이라도 밟고 있는 바닥에 따라 소리가 달라야 합니다. RandomAudioPlayer는 기본 클립 묶음(clips[])과, 타일별 대체 묶음(overrides[])을 함께 들고 있습니다.

```
// 구조
public AudioClip[] clips;              // 기본
public TileOverride[] overrides;       // { TileBase tile; AudioClip[] clips; }
public bool  randomizePitch = false;
public float pitchRange     = 0.2f;


// Awake 에서 Dictionary<TileBase, AudioClip[]> 로 변환해 둔다


// PlayRandomSound(TileBase surface = null)
AudioClip[] source = clips;
if (surface != null && m_LookupOverride.TryGetValue(surface, out temp)) source = temp;
if (randomizePitch) m_Source.pitch = Random.Range(1f - pitchRange, 1f + pitchRange);
m_Source.PlayOneShot(source[Random.Range(0, source.Length)]);
```

타일 쪽에는 AudioSurface 컴포넌트를 둡니다. 이 컴포넌트는 public TileBase tile 필드 하나만 가진 표식용 컴포넌트입니다. PhysicsHelper.FindTileForOverride가 접지 콜라이더에서 밟고 있는 타일을 찾아 주고, PlayerCharacter가 그것을 m_CurrentSurface에 넣어 두었다가 발소리·착지음에 넘깁니다.

Ellen에게는 다섯 개의 RandomAudioPlayer가 붙습니다 — footstep · landing · hurt · meleeAttack · rangedAttack. 이 중 착지음은 조건이 하나 더 있습니다. CheckForGrounded는 직전에 공중이었고 m_MoveVector.y < −1.0f일 때만 착지음을 울립니다. 작은 턱을 내려올 때마다 소리가 나지 않게 하는 처리입니다.

VFXController — 이펙트도 풀에서 꺼낸다

이펙트는 Instantiate하지 않고 VFXInstancePool에서 꺼내 씁니다. 수명(VFX.lifetime, 기본 1초)이 끝나면 풀로 돌아갑니다. VFX 설정에도 타일별 오버라이드(vfxOverride)가 있어, 같은 "DustPuff"가 바닥 재질에 따라 다른 프리팹으로 나올 수 있습니다.

- **VFX —** 이펙트 설정. prefab, lifetime(기본 1), vfxOverride[](타일별 대체 프리팹).
- **VFXInstance / VFXInstancePool —** 풀 항목과 풀. VFXInstance 는 PoolObject<VFXInstancePool, VFXInstance>를 상속하면서 IComparable<VFXInstance>도 구현하고, Animation · AudioSource · ParticleSystem[]을 캐시한다.
- **VFXController —** 싱글턴. vfxConfig[]를 이름 해시로 찾아 Trigger(name, position, startDelay, flip, parent, tileOverride)로 띄우고, 실행 중인 것과 대기 중인 것을 각각 PriorityQueue 로 관리한다.

PlayerCharacter.PlayFootstep()이 VFXController.Instance에 "DustPuff"를 요청하는 것이 대표적인 사용 예입니다. startDelay를 주어 조금 뒤에 띄우는 것이 가능하므로, 애니메이션의 타이밍과 이펙트를 맞출 수 있습니다.

7-10. 다국어와 타임라인

다국어 (Localization)

- **OriginalPhrases / TranslatedPhrases —** 키와 원문의 모음, 그리고 한 언어의 번역 모음(언어 이름 + 번역 문자열).
- **Translator —** 싱글턴. phrases 목록과 현재 언어 인덱스를 들고 인덱서 this[key]로 번역을 돌려준다. SetLanguage 는 인덱스·언어 이름·TranslatedPhrases 객체 세 가지 오버로드를 가진다.

- **TranslatedText** — TextMeshProUGUI 에 붙어 phraseKey 로 번역을 찾아 넣는다. setTextOnStart(기본 true)이면 Start 에서 자동으로 한 번 적용된다.

Editor 폴더에 PhraseDrawer · TranslatedPhrasesEditor · TranslatorEditor 세 개의 에디터 확장이 함께 들어 있어, 번역 표를 인스펙터에서 편집할 수 있습니다. Localization 그룹은 전체 7개 파일 359줄로 키트에서 가장 작은 하위 시스템입니다.

타임라인 (Timeline)

컷신은 Unity Timeline의 PlayableDirector로 재생합니다. DirectorTrigger가 그 시작을 맡습니다.

항목	내용
DirectorTrigger	triggeringGameObject가 들어오면 director를 재생한다. triggerType은 Once / Everytime 두 가지이며, Once이면 m_AlreadyTriggered를 세우고 그 상태를 IDataPersister로 저장한다
OverrideAlreadyTriggered	외부에서 "이미 발생한 것으로 처리"하게 만드는 메서드. HubDoor가 열쇠를 확인할 때 이 메서드로 열쇠 안내 컷신을 건너뛰게 한다
ScrollingText 트랙	자작 Timeline 트랙. ScrollingTextTrack · ScrollingTextClip · ScrollingTextBehaviour · ScrollingTextMixerBehaviour로 구성되며, message를 startDelay 후 한 글자씩 드러내고 holdDelay만큼 유지한다
PrewarmDirector	Timeline을 미리 한 번 돌려 두어 첫 재생 때의 멈춤을 줄이는 유틸리티

ScrollingText는 Timeline에 직접 트랙 타입을 추가하는 예제로서 가치가 있습니다. Timeline 그룹 전체가 7개 파일 228줄이므로, 트랙 하나를 만드는 데 필요한 최소 구성을 읽기에 적당한 분량입니다.

8. 기술 명세 (Technical Specification)

8-1. 환경과 네임스페이스

Unity 버전	Unity 2017.3 이상 (Unity Learn 공식 요구 사항)
언어	C#
입력	Unity 새 입력 시스템 (UnityEngine.InputSystem) — 키보드와 게임패드를 함께 바인딩

카메라	Unity.Cinemachine (TransitionPoint가 참조)
UI 텍스트	TextMeshPro (TextMeshProUGUI)
컷신	Unity Timeline (PlayableDirector) + 자작 ScrollingText 트랙
코드 규모	126개 파일, 11,552줄
네임스페이스	Gamekit2D — 121개 파일 (키트 본체) BTAI — 2개 파일에서 선언 (AI/BT.cs, AI/SendSignal.cs). MissileGolem.cs와 SpitterBT.cs가 using BTAI로 사용한다 네임스페이스 없음 — 3개 파일 (AI/MissileGolem.cs, Utility/OpenURL.cs, Utility/UIAutoscroll.cs)

8-2. 디렉터리 구조와 코드 규모

Scripts 폴더를 13개 그룹으로 나누어 센 결과입니다. 파일 수와 줄 수의 합이 각각 126과 11,552입니다.

그룹	파일	줄	담당
AI/	7	1,304	BTAI 행동 트리 프레임워크(BT.cs 710줄), 보스 MissileGolem, 보스 빔·발사체
Audio/	4	378	RandomAudioPlayer, AudioSurface, BackgroundMusicPlayer, 믹서 슬라이더
Character/MonoBehaviours/	11	2,762	PlayerCharacter(810줄), EnemyBehaviour(588줄), CharacterController2D, Damager, Damageable, PlayerInput, InputComponent, Bullet, BulletPool, SpitterBT, VFXController
Character/StateMachineBehaviours/	19	657	SceneLinkedSMB와 Player 15개 · Enemies 4개의 상태별 동작
Character/Editor/	5	772	PlayerCharacter · PlayerInput · Damager · Damageable · CharacterStateSetter의 인스펙터 확장
Core/	13	718	ObjectPool, InventoryController, InventoryItem, Checkpoint, 상호작용 6종, LayerMask 확장
Effect/	7	698	WaterArea(454줄), Grenade, GunnerLightning, GunnerShieldEffect, DropShadow, 보스 문 상태

그룹	파일	줄	담당
Localization/	7	359	Translator, OriginalPhrases, TranslatedPhrases, TranslatedText와 에디터 3종
Objects/	13	1,396	MovingPlatform, PlatformCatcher, PressurePad, Pushable, HubDoor, EnemySpawner, HealthPickup, WeaponPickup과 에디터 3종
SceneManagement/	15	1,460	SceneController, PersistentDataManager, IDataPersister, TransitionPoint, ScreenFader, CharacterStateSetter와 에디터 4종
Timeline/	7	228	DirectorTrigger와 ScrollingText 트랙 일식
UI/	6	260	HealthUI, KeyUI, InputDisplayUI, StartUI, OptionUI, MenuActivityController
Utility/	12	560	PhysicsHelper, Helpers, PriorityQueue, AutoCameraSetup, OpenURL, UIAutoscroll 등
합계	126	11,552	

8-3. 주요 수치 — 코드 기본값과 Reference Guide 권장값

이 표가 이 문서의 핵심입니다. 왼쪽은 스크립트에서 필드를 초기화한 값, 즉 "새 오브젝트에 컴포넌트를 처음 붙였을 때 보이는 값"입니다. 오른쪽은 Reference Guide가 제시하는 Ellen 프리팹의 설정값입니다. 두 값이 다른 항목은 비고에 ★로 표시했습니다. 직접 만든 캐릭터가 공식 Ellen과 다르게 움직인다면, 대부분 이 표의 ★ 항목 때문입니다.

PlayerCharacter (Ellen)

필드	코드 기본값	Reference Guide 권장값	단위 · 의미	비고
maxSpeed	10	7	유닛/초 — 지상·공중 최대 이동 속도	★ 코드 기본값이 1.4배 빠르다
groundAcceleration	100	100	유닛/초² — 입력이 있을 때의 가속	같음

필드	코드 기본값	Reference Guide 권장값	단위 · 의미	비고
groundDeceleration	100	100	유닛/초² — 입력이 없을 때의 감속	같음
pushingSpeedProportion	초기화 없음 (0)	0.5	비율 0~1 — 상자를 밀 때 적용할 최대 속도 비율	★ 코드 기본값 0이면 상자를 밀 때 전혀 움직이지 않는다
airborneAccelProportion	초기화 없음 (0)	(미제시)	비율 0~1 — 공중 가속 비율	[Range(0,1)]
airborneDecelProportion	초기화 없음 (0)	(미제시)	비율 0~1 — 공중 감속 비율	[Range(0,1)]
gravity	50	38	유닛/초² — 매 프레임 m_MoveVector.y에서 빼는 중력	★ 코드 기본값이 더 무겁다
jumpSpeed	20	16.5	유닛/초 — 점프 시작 상승 속도	★
jumpAbortSpeedReduction	100	100	유닛/초² — 버튼을 일찍 뗐을 때 상승 속도를 깎는 비율	같음
hurtJumpAngle	45	45	도 — 피격 시 튕겨 나가는 각도	같음. [Range(0.001, 89.999)]
hurtJumpSpeed	5	8	유닛/초 — 피격 시 튕겨 나가는 속도	★ 권장값이 더 크게 튕긴다
flickeringDuration	0.1	0.05	초 — 피격 후 점멸 간격	★ 권장값이 두 배 빠르게 깜빡인다
meleeAttackDashSpeed	5	15	유닛/초 — 근접 공격 시 앞으로 미끄러지는 속도	★ 권장값이 3배. 코드 기본값으로는 돌진감이 거의 없다

필드	코드 기본값	Reference Guide 권장값	단위 · 의미	비고
dashWhileAirborne	false	(미제시)	공중 근접 공격에서도 미끄러질지	bool
shotsPerSecond	1	2	발/초 — Start()에서 역수를 취해 발사 간격으로 바꾼다	★ 권장값이 두 배 빠르다
bulletSpeed	5	30	유닛/초 — 총알의 수평 속도	★ 권장값이 6배. 코드 기본값으로는 총알이 걷는 속도로 날아간다
holdingGunTimeoutDuration	10	(미제시)	초 — 쏘지 않아도 총 든 자세를 유지하는 시간	
rightBulletSpawnPointAnimated	true	(미제시)	오른쪽 스폰 지점이 애니메이션으로 움직이는지	true면 오른쪽을 기준으로 왼쪽 좌표를 x만 반전해 만든다
spriteOriginallyFacesLeft	초기화 없음 (false)	(미제시)	원본 스프라이트가 왼쪽을 보는지	flipX 계산과 총알 스폰 지점 선택의 기준

CharacterController2D

필드	코드 기본값	Reference Guide 권장값	의미
groundedLayerMask	초기화 없음 (Nothing)	Platform	접지로 인정할 레이어. 비워 두면 영원히 공중 상태가 된다
groundedRaycastDistance	0.1	0.1	아래쪽으로 지면을 확인하는 추가 거리. 실제 사거리는 capsule.size.x × 0.5 + 이 값 × 2

기타 컴포넌트의 코드 기본값

Reference Guide가 별도의 권장값을 제시하지 않은 항목입니다. Damager·Damageable의 값은 4-3절, 보스의 값은 5-4절에 이미 표로 두었으므로 여기서는 생략합니다. "초기화 없음"은 스크립트에 초기값이 없어 C#의 기본값(숫자 0, bool false, LayerMask Nothing)이 들어간다는 뜻이며, 그대로 두면 동작하지 않는 항목이 여럿 있습니다.

컴포넌트	필드	코드 기본값	의미
EnemyBehaviour	speed	초기화 없음 (0)	이동 속도. 프리팹에서 반드시 채워야 한다
EnemyBehaviour	gravity	10	중력 가속도 (유닛/초²)
EnemyBehaviour	viewDirection	0	시야 중심 각도 (도). [Range(0, 360)]
EnemyBehaviour	viewFov	초기화 없음 (0)	시야 전체 폭 (도). 판정에는 절반을 쓴다. 0이면 아무것도 못 본다
EnemyBehaviour	viewDistance	초기화 없음 (0)	시야 거리 (유닛). 0이면 아무것도 못 본다
EnemyBehaviour	timeBeforeTargetLost	3	시야에서 벗어나도 쫓는 시간 (초)
EnemyBehaviour	meleeRange	3	근접 공격 사거리 (유닛)
EnemyBehaviour	flickeringDuration	초기화 없음 (0)	피격 점멸 간격 (초). 0이면 점멸이 보이지 않는다
EnemyBehaviour	(발사 속력)	30 (const)	GetProjectilVelocity 안의 const float projectileSpeed. 인스펙터에 노출되지 않는다
MovingPlatform	speed / platformType / isMovingAtStart / localNodes · waitTimes	1.0 / BACK_FORTH / true / 길이 1	이동 속도(유닛/초, Walkthrough 예시값은 2), 경로 순환 방식, 시작 시 이동 여부, 경로 노드와 노드별 대기
PressurePad	(지연 프레임)	2 (static int)	DELAYEDFRAME_COUNT. 이 프레임 수를 넘겨야 OnPressed / OnRelease가 발생한다

컴포넌트	필드	코드 기본값	의미
ObjectPool	initialPoolCount	10	처음 만들어 둘 항목 수
EnemySpawner	spawnArea	1.0	소환 위치의 좌우 폭 (유닛). ±spawnArea × 0.5 범위에서 무작위
HealthPickup	healthAmount	1	회복량
Bullet	destroyWhenOutOfView / timeBeforeAutodestruct	true / −1.0	화면 밖으로 나가면 풀로 반납할지 / 자동 소멸 시간(초, 음수면 비활성)
ScreenFader	fadeDuration	1	페이드 한 번에 걸리는 시간 (초)
WaterArea	size / pointPerUnits / dampening · tension · neighbourTransfer / sortingLayerOrder / (입수 충격)	(6, 2) / 5 / 0.93 · 0.025 · 0.03 / 3 / 0.2	수면 크기(유닛), 유닛당 분할 점수(분할 수 = ceil(size.x × (값 − 1)) + 1), 물결 스프링 모델의 감쇠·복원·이웃 전달 계수, 메시 정렬 순서, OnTriggerEnter2D의 testForce
RandomAudioPlayer	randomizePitch / pitchRange	false / 0.2	피치 무작위화 여부와 범위 (1 ± pitchRange)
Grenade	timer / explosionTimer	1 / 3	폭발까지의 시간과 폭발 효과 유지 시간 (초)
InventoryItem	disableOnEnter	false	주운 뒤 오브젝트를 끌지. Reset()은 CircleCollider2D의 radius를 5로 설정한다
TranslatedText	setTextOnStart	true	Start에서 번역을 자동 적용할지
KeyUI / HealthUI	(아이콘 간격)	0.041 (const)	k_KeyIconAnchorWidth / k_HeartIconAnchorWidth. 아이콘 하나당 앵커를 옮기는 폭

8-4. enum 식별자 전수

문서나 수업 자료에서 enum 값을 적을 때는 아래 식별자를 그대로 써야 합니다. 통칭과 실제 식별자가 다른 경우가 있습니다.

enum	값	쓰이는 곳
BTAI.BTState	Failure, Success, Continue, Abort	행동 트리 노드의 Tick() 반환값. Abort는 공식 Advanced Topics 문서에 나오지 않는다
MovingPlatform.MovingPlatformType	BACK_FORTH, LOOP, ONCE	이동 플랫폼의 경로 순환 방식
PressurePad.ActivationType	ItemCount, ItemMass	압력판의 작동 기준. 두 번째 값은 Mass가 아니라 ItemMass다
TransitionPoint.TransitionType	DifferentZone, DifferentNonGameplayScene, SameScene	전환의 종류
TransitionPoint.TransitionWhen	ExternalCall, InteractPressed, OnTriggerEnter	전환이 시작되는 계기
SceneTransitionDestination.DestinationTag	A, B, C, D, E, F, G	전환 지점과 도착 지점을 짝지우는 태그. 7개뿐이다
DataSettings.PersistenceType	DoNotPersist, ReadOnly, WriteOnly, ReadWrite	영속화 방식
ScreenFader.FadeType	Black, Loading, GameOver	페이드 화면의 종류
DirectorTrigger.TriggerType	Once, Everytime	Timeline 재생 트리거의 반복 여부
CharacterStateSetter.ParameterSetter.ParameterType	Bool, Float, Int, Trigger	Animator 파라미터의 종류

9. 알려진 제약과 주의점

코드를 전수 분석하면서 확인한, 문서에 적혀 있지 않은 사항들입니다. 학습용 프로젝트에 남아 있는 미완성 코드와, 이름이나 주석과 실제 동작이 다른 부분이 섞여 있습니다. 수업에서는 이 표를 "남의 코드를 읽을 때 무엇을 의심해야 하는가"의 예시로 쓸 수 있습니다.

9-1. 동작하지 않거나 쓰이지 않는 코드

No.	위치	확인된 사실	영향 · 대응
1	SceneManagement/SavedData.cs	문자열·bool·int·float·Vector2용 Dictionary 5개를 선언만 하고 초기화하지 않는다. Get/Set이 모두 그 Dictionary를 바로 쓰므로 호출하면 NullReferenceException이 난다. 프로젝트 전체에서 참조 0건.	사실상 죽은 코드. 영속화는 PersistentDataManager + IDataPersister가 담당한다. 쓸 생각이라면 Dictionary 5개를 먼저 생성해야 한다.
2	SceneManagement/Editor/ ParameterSetterDrawer.cs	FindPropertyRelative("animator")로 찾는 animator가 ParameterSetter에는 없다 — 바깥 클래스 CharacterStateSetter의 필드다. m_AnimatorProp이 null이 되고 GetPropertyHeight가 0f를 반환해 인스펙터에 아무것도 그려지지 않는다.	CharacterStateSetter의 Parameter Setters를 인스펙터에서 편집할 수 없다. 드로어의 경로를 바깥 animator로 바꾸거나 ParameterSetter에 animator 필드를 추가해야 한다.
3	AI/GunnerBeamController.cs	다섯 개의 콜백이 모두 주석 처리된 빈 클래스다. Unity가 새 SMB를 만들 때 주는 템플릿 그대로다.	붙여 두어도 아무 일도 일어나지 않는다. 빔 제어는 MissileGolem의 행동 트리와 GunnerBeam이 담당한다.
4	AI/MissileGolem.cs Disabled() / Enabled()	두 메서드가 본문 없이 비어 있다. 행동 트리는 같은 이름의 Animator 트리거만 걸 뿐이다.	라운드 전환 시점에 추가 처리를 넣을 자리로 남겨 둔 빈 훅으로 읽으면 된다.

No.	위치	확인된 사실	영향 · 대응
5	Character/MonoBehaviours/PlayerCharacter.cs OnHurt() / OnDie()	코드에서 호출하는 곳이 한 군데도 없다(선언부 외 참조 0건).	Damageable의 OnTakeDamage / OnDie에 인스펙터로 연결해야 피격·사망 처리가 동작한다. 피격 반응이 없다면 이 연결을 먼저 확인한다.

9-2. 이름이나 문서와 실제 동작이 다른 것

No.	위치	확인된 사실	영향 · 대응
6	AI/BT.cs — Selector(bool shuffle)	생성자에서 children을 섞지만, 그 시점에는 OpenBranch가 아직 호출되지 않아 children이 비어 있다. shuffle 인자는 아무것도 섞지 않는다.	BT.Selector(true)는 BT.Selector(false)와 같다. 무작위가 필요하면 RandomSequence를 쓴다.
7	AI/BT.cs — While	조건이 true인 동안 항상 Continue, false가 되면 ResetChildren() 후 Failure를 반환한다. Success를 내는 경로가 아예 없다.	While을 Sequence의 자식으로 두면 영원히 다음으로 넘어가지 않는다. 부모가 Failure를 무시하는 Block 계열(Root, Repeat 등)이어야 한다. MissileGolem이 While을 Repeat 안에 둔 이유다.
8	Utility/PriorityQueue.cs SiftDown / SiftUp	SiftDown은 부모를 따라 올라가며 원소를 루트 쪽으로 올리고, SiftUp은 자식을 따라 내려가며 잎 쪽으로 내린다. 이름과 동작 방향이 반대다.	동작 자체는 정상적인 이진 힙이다(Push는 SiftDown으로, Pop은 SiftUp으로). 이름만 보고 읽으면 혼란하다.
9	Utility/PriorityQueue.cs Contains	List<T>의 선형 탐색이다. 주석의 "A fast priority queue or heap implementation"과 달리 이 메서드만은 O(n).	VFXController가 쓰는 대기열은 짧으므로 실제 문제는 되지 않는다.

No.	위치	확인된 사실	영향 · 대응
10	UI/InputDisplayUI.cs	이름 때문에 F12 디버그 창으로 오해하기 쉬우나, 실제로는 바인딩 경로를 읽어 "Space - Jump" 같은 조작법 안내 문구를 만드는 UI다. F12 창은 PlayerInput.OnGUI가 그린다.	조작 안내는 이 파일을, 디버그 토글은 PlayerInput.cs를 본다.

9-3. 설정이나 상황에 따라 문제가 되는 것

No.	위치	확인된 사실	영향 · 대응
11	Objects/HubDoor.cs Update()	Update()가 매 프레임 CheckInventory()를 호출하고, 열쇠를 전부 모으면 그 조건이 계속 참이므로 onUnlocked가 프레임마다 다시 발생한다.	onUnlocked에 효과음이나 1회성 연출을 연결하면 매 프레임 재생된다. 이미 열렸는지 표시하는 bool을 두거나 받는 쪽에서 중복 실행을 막아야 한다.
12	Objects/EnemySpawner.cs Start()	실제 소환 수는 Min(total − 누적, concurrent)로 계산하는데, 누적 카운터에는 concurrentEnemiesToBeSpawned를 그대로 더한다.	total이 concurrent보다 작으면 실제보다 많이 소환한 것으로 기록되어 적이 예정보다 적게 나온다. total ≥ concurrent로 두면 피할 수 있다.
13	Effect/WaterArea.cs 물결 계산	velocity = dampening × velocity + force, currentHeight += velocity 두 식에 Time.deltaTime이 곱해져 있지 않다. 프레임마다 같은 양이 누적된다.	60fps와 144fps에서 출렁임 속도가 다르게 보인다. 고정하려면 두 식에 Time.deltaTime을 반영하고 세 계수를 다시 조율해야 한다. 수면 연출이라 게임플레이에는 영향이 없다.
14	AI/MissileGolem.cs NextRound()	라운드 전환 음악을 round == 2와 round == 3만 처리한다(round는 증가 후의 값).	rounds를 4개 이상으로 늘리면 4라운드부터 전환 클립이 재생되지 않는다. 조건을 배열 인덱스 기반으로 바꾸고 클립도 배열로 만들어야 한다.

No.	위치	확인된 사실	영향 · 대응
15	Core/InteractOnButton2D.cs ExecuteOnEnter()	부모의 ExecuteOnEnter는 OnEnter 후 inventoryChecks[]를 확인하는데, 재정의한 버전은 플래그만 세우고 OnEnter만 부른다.	InteractOnButton2D에 inventoryChecks를 채워도 확인되지 않는다. 열쇠 확인이 필요하면 InteractOnTrigger2D를 쓰거나 OnButtonPress 쪽에서 직접 확인한다.
16	Character/MonoBehaviours/ PlayerCharacter.cs PlayerInstance	Awake()에서 조건 없이 s_PlayerInstance = this로 덮어쓰고, 파괴될 때 null로 되돌리지 않는다.	씬에 PlayerCharacter가 둘 이상이면 나중에 Awake된 쪽이 조용히 이긴다. 씬을 떠난 뒤 파괴된 오브젝트를 가리킬 수도 있다. PlayerInput은 반대로 둘 이상이면 Awake에서 UnityException을 던진다.

10. 확장 가이드

키트의 구조를 그대로 따라 새 요소를 추가하는 절차입니다. Advanced Topics의 설명과 실제 코드 패턴을 맞추어 정리했습니다. 각 절차는 "코드를 새로 쓰는 양을 최소로 한다"는 키트의 방침을 유지하는 순서로 적었습니다.

10-1. 새 적 추가

EnemyBehaviour를 재사용하고 조종 주체만 새로 만드는 것이 기본 방침입니다. 공격·시야·피격·사망은 이미 EnemyBehaviour에 있으므로 다시 쓸 필요가 없습니다.

단계	작업	주의할 점
1	기존 Chomper 또는 Spitter 프리팹을 복제한다. 근접형이면 Chomper, 원거리형이면 Spitter를 고른다.	처음부터 빈 오브젝트로 만들면 CharacterController2D · Damageable · Damager · Animator · SpriteRenderer · RandomAudioPlayer의 연결을 모두 다시 해야 한다.

단계	작업	주의할 점
2	EnemyBehaviour의 필드를 채운다.	speed · viewFov · viewDistance · flickeringDuration은 코드에 초기화가 없어 0이다. 반드시 채워야 한다. viewFov는 시야 전체 폭이므로 전방 90도를 원하면 90을 적는다.
3	Damageable.startingHealth로 체력을, Damager의 damage · offset · size · hittableLayers로 공격을 설정한다.	hittableLayers에 Player를 넣지 않으면 적이 플레이어를 때리지 못한다.
4	Damageable의 OnTakeDamage에 EnemyBehaviour.Hit을, OnDie에 EnemyBehaviour.Die를 인스펙터에서 연결한다.	9-1절 5번 항목. 이 연결이 없으면 피격 점멸도 사망 처리도 일어나지 않는다.
5	조종 주체를 정한다. (가) 상태 기계: Animator 상태에 SceneLinkedSMB<EnemyBehaviour>를 상속한 SMB를 붙인다. (나) 행동 트리: SpitterBT를 본뜬 MonoBehaviour를 만들고 OnEnable에서 트리를 조립해 Update에서 Tick한다.	(가)는 전이 조건을 Animator 창에서 눈으로 보고 고칠 수 있다. (나)는 판단 순서를 코드 한 곳에서 읽을 수 있다. 수업에서는 둘을 같은 적으로 만들어 비교해 보게 하는 과제가 효과적이다.
6	SMB 방식이면, 적 쪽 MonoBehaviour의 Start()에서 SceneLinkedSMB<EnemyBehaviour>.Initialise(animator, enemyBehaviour)를 호출해 참조를 주입한다.	이 호출이 없으면 SMB의 m_MonoBehaviour가 null이어서 첫 Update에서 예외가 난다.
7	Animator 파라미터를 맞춘다 — Spotted · Shooting · TargetLost · MeleeAttack · Hit · Death · Grounded.	EnemyBehaviour가 이 일곱 개 이름을 StringToHash로 고정해 두었다. 파라미터 이름이 다르면 트리거가 아무 상태에도 닿지 않는다.
8	소환기로 내보낼 적이면 EnemySpawner의 prefab에 등록하고 initialPoolCount를 예상 동시 등장 수에 맞춘다.	풀은 줄어들지 않으므로(7-6절) 넉넉히 잡아도 손해가 크지 않다. 다만 total ≥ concurrent로 두어야 한다(9-3절 12번).

10-2. 새 상호작용 오브젝트 추가

"조건이 만족되면 무엇을 한다"는 형태의 오브젝트는 대부분 코드를 쓰지 않고 만들 수 있습니다. 조건 쪽과 결과 쪽에 이미 있는 컴포넌트를 골라 UnityEvent로 잇는 것이 기본 절차입니다.

조건 (발생 쪽)	쓰는 컴포넌트	발생하는 이벤트
영역에 들어오면 / 나가면	InteractOnTrigger2D	OnEnter / OnExit
영역 안에서 E를 누르면	InteractOnButton2D	OnButtonPress (+ OnEnter / OnExit)
물건이나 플레이어가 올라가면	PressurePad	OnPressed / OnRelease
특정 열쇠를 가지고 있으면	InventoryController.InventoryChecker	OnHasItem / OnDoesNotHaveItem
열쇠를 전부 모으면	HubDoor	onUnlocked
피해를 입히면 / 입으면	Damager / Damageable	OnDamageableHit, OnNonDamageableHit / OnTakeDamage, OnDie, OnHealthSet, OnGainHealth
도착 지점에 닿으면	SceneTransitionDestination	OnReachDestination
Timeline이 시작되면 / 끝나면	DirectorTrigger	OnDirectorPlay / OnDirectorFinish

결과 (받는 쪽)	연결하는 대상	예
애니메이션 재생	Animator.Play(string)	문 열기 — 문자열 "DoorOpening"
오브젝트 켜고 끄기	GameObject.SetActive(bool)	숨은 발판 드러내기
플랫폼 움직이기 / 멈추기	MovingPlatform.StartMoving() / StopMoving() / ResetPlatform()	압력판을 밟는 동안만 움직이는 플랫폼
소리 재생	RandomAudioPlayer.PlayRandomSound()	장치 작동음
피해 판정 켜고 끄기	Damager.EnableDamage() / DisableDamage()	주기적으로 켜지는 함정
무기 사용 허용	PlayerInput.EnableMeleeAttacking() / EnableRangedAttacking()	무기 획득 지점

결과 (받는 쪽)	연결하는 대상	예
Timeline 재생	PlayableDirector.Play()	컷신 시작
구역 이동	SceneController.TransitionToScene(TransitionPoint)	외부 호출 방식의 전환 (TransitionWhen.ExternalCall)

그래도 코드가 필요할 때

· 위 조합으로 표현되지 않는 조건이라면, MonoBehaviour를 하나 만들고 public UnityEvent 필드를 두어 같은 방식에 합류시키는 것이 키트의 관례다. 조건 판정만 코드로 하고, 결과는 여전히 인스펙터에서 잇는다.

· 영속화가 필요하면(한 번 열린 문은 다시 와도 열려 있어야 한다면) IDataPersister를 구현하고, OnEnable에서 PersistentDataManager.RegisterPersister(this), OnDisable에서 UnregisterPersister(this)를 부른다. 상태가 바뀔 때 PersistentDataManager.SetDirty(this)를 부르면 즉시 저장된다.

· Data Tag를 비워 두면 기본값이 새 GUID이므로 씬을 넘나들며 상태가 이어지지 않는다. 직접 의미 있는 문자열을 적어 둔다.

10-3. 새 오브젝트 풀 만들기

자주 생성·파괴되는 것이 있으면 풀을 만듭니다. 풀 클래스와 항목 클래스를 짝으로 선언하는 것이 ObjectPool의 사용법입니다. 제네릭 인자가 자기 자신을 포함하므로(F-bounded 패턴) 선언이 조금 낯설게 보입니다.

```
// (가) 꺼낼 때 넘길 정보가 없는 경우
public class MyPool : ObjectPool<MyPool, MyItem> { }


[System.Serializable]
public class MyItem : PoolObject<MyPool, MyItem>
{
    public Transform transform;               // instance 에서 캐시할 컴포넌트
    protected override void SetReferences()   // 생성 직후 한 번
    { transform = instance.transform; }
    public override void WakeUp()  { instance.SetActive(true);  }   // Pop 될 때
    public override void Sleep()   { instance.SetActive(false); }   // Push 될 때
}


// (나) 꺼낼 때 정보를 넘기는 경우 – 세 번째 인자가 그 타입
//     MyPool : ObjectPool<MyPool, MyItem, Vector2>
//     MyItem : PoolObject<MyPool, MyItem, Vector2>  →  WakeUp(Vector2 position)
```

단계	작업	주의할 점
1	풀 클래스와 항목 클래스를 위 형태로 선언한다. 꺼낼 때 위치 등을 넘기려면 TInfo 버전을 쓴다.	PoolObject는 MonoBehaviour가 아니다. [System.Serializable]을 붙여야 인스펙터에서 보인다.

단계	작업	주의할 점
2	SetReferences에서 instance의 컴포넌트를 캐시한다.	매번 GetComponent를 부르지 않기 위한 자리다. 이것이 PoolObject의 주요 존재 이유다.
3	WakeUp에서 활성화·위치 설정을, Sleep에서 비활성화를 한다.	Sleep은 생성 직후에도 한 번 불린다. 따라서 "처음부터 꺼져 있는 상태"를 Sleep이 보장한다.
4	풀 컴포넌트를 씬 오브젝트에 붙이고 prefab과 initialPoolCount를 지정한다.	Start()가 initialPoolCount만큼 미리 만든다. 항목의 instance는 풀 오브젝트의 자식이 된다.
5	쓰는 쪽에서 Pop() 또는 Pop(info)로 꺼낸다.	반환값이 항목 객체(MyItem)다. GameObject가 아니라 instance 필드로 접근한다.
6	항목 자신이 일이 끝났을 때 ReturnToPool()을 호출한다.	Bullet은 화면 밖으로 나가거나 timeBeforeAutodestruct가 지나면 스스로 반납한다. Enemy는 사망 이벤트에서 코루틴으로 removalDelay 후 반납한다. 반납을 잊으면 풀이 계속 커진다.
7	필요하면 Pop / Push를 override해 추가 처리를 넣는다.	EnemySpawner가 Push를 override해 현재 소환 수를 줄이고 소환 타이머를 다시 켜는 것이 그 예다.

10-4. 그 밖의 확장 지점

하려는 일	건드릴 곳
새 타일의 발소리를 다르게 하기	타일 오브젝트에 AudioSurface를 붙여 TileBase를 지정하고, 해당 RandomAudioPlayer의 overrides[]에 그 TileBase와 클립 묶음을 추가한다
새 이펙트 추가	VFXController의 vfxConfig[]에 VFX 항목을 추가하고(prefab, lifetime, vfxOverride) 코드에서 VFXController.Instance.Trigger("이름", ...)으로 띄운다
키 바인딩 변경	PlayerInput.cs의 InputButton · InputAxis 생성자 인자를 바꾼다. InputDisplayUI의 조작 안내는 바인딩을 읽으므로 자동으로 따라온다
새 열쇠와 잠긴 문 추가	InventoryItem.inventoryKey → InventoryController.inventoryEvents[].key → KeyUI.keyNames[] → HubDoor.requiredInventoryItemKeys[] 네 곳의 문자열을 정확히 일치시킨다. unlockStateSprites의 길이도 키 개수에 맞춘다

하려는 일	건드릴 곳
새 구역 추가	Template 씬을 복제해 레벨을 만들고, 양쪽 씬에 TransitionPoint와 SceneTransitionDestination을 두고 DestinationTag(A~G)를 맞춘다. 새 씬 이름은 Build Settings에 등록한다
보스 라운드 늘리기	MissileGolem.rounds 배열을 늘린다. 라운드 전환 음악은 round 2·3만 처리하므로 함께 수정해야 한다(9-3절 14번)
새 Timeline 트랙 만들기	Timeline/ScrollingText의 네 파일(Track · Clip · Behaviour · MixerBehaviour)을 본으로 삼는다. 전체 228줄이므로 최소 구성을 읽기에 적당하다
번역 추가	OriginalPhrases에 키와 원문을, TranslatedPhrases에 번역을 추가하고, 표시할 TextMeshProUGUI에 TranslatedText를 붙여 phraseKey를 지정한다

11. 참고 자료

11-1. Unity Learn 공식 자료

2D Game Kit 과정 전체 — https://learn.unity.com/course/2d-game-kit

이 한 주소 아래에 다섯 개의 튜토리얼이 순서대로 들어 있습니다. 난이도 Beginner, 총 소요 시간 4시간 25분, Unity 2017.3 이상.

- ① Get Started with the 2D Game Kit — 내려받기와 프로젝트 구조
- ② 2D Game Kit Walkthrough — 레벨 제작 10 단계 실습
- ③ Video: 2D Game Kit Walkthrough — 위 실습의 45 분 영상 강의 (8 장)
- ④ 2D Game Kit Reference Guide — 컴포넌트 설명과 권장 설정값 10 개 절
- ⑤ 2D Game Kit Advanced Topics — 내부 구조 설명 6 개 절

11-2. 프로젝트 내부 자료

위치	내용
Assets > 2DGameKit > Documentation	키트에 함께 들어 있는 문서 폴더
Assets > 2DGameKit > Scripts	분석 대상 C# 소스 126개 파일, 11,552줄
Assets > 2DGameKit > Prefabs > Interactables	MovingPlatform, PressurePad, Door 등 상호작용 오브젝트
Assets > 2DGameKit > Prefabs > Enemies	Chomper, Spitter

위치	내용
Assets > 2DGameKit > Prefabs > SceneControl	TransitionStart, TransitionEnd, TransitionDestination
Assets > 2DGameKit > Prefabs > UIPrefabs	UI 프리팹
Assets > 2DGameKit > Art > Sprites > Environment	장식용 환경 스프라이트
Assets > 2DGameKit > Art > Animations > Animation Clips	애니메이션 클립
Assets > 2DGameKit > Audio > Interactables	상호작용 오브젝트 효과음
Assets > 2DGameKit > Scenes	ExampleScene, Start, Zone1~Zone5, Template, UIMenus

11-3. 문서 작성 기준

GDD의 절 구성은 Unity Learn Game Development Pathway(Unit 4)의 Game Design Document 템플릿을 따랐습니다. 게임 내용과 학습 과정 정보는 Unity Learn의 2D Game Kit 과정 5개 튜토리얼에 근거하고, 컴포넌트 구조·필드명·수치·enum 식별자·실행 순서는 모두 프로젝트의 C# 소스 126개 파일을 직접 읽어 확인했습니다. 원본 프로젝트의 저작권은 Unity Technologies에 있으며, 이 문서는 원광대학교 게임콘텐츠학과「게임프로그래밍」교육 목적으로 작성되었습니다.